

软件安全

第1章 软件安全概述

徐国胜
北京邮电大学

内容安排

- 课程介绍
- 基本概念
 - 1 软件与软件安全
 - 2 软件安全
 - 3 漏洞
 - 4 软件风险
- 软件安全基础
 - 1 计算机引导
 - 2 Win32内存体系
 - 3 PE文件结构
 - 4 进程空间分区
 - 5 系统栈工作
 - 6 缓冲区溢出

内容安排

- 课程介绍
 - 1 教师、教材与交流
 - 2 课程目标
 - 3 理论课介绍
 - 4 实验课介绍
 - 5 考核要点
 - 6 提交要求
 - 7 资讯范例
 - 8 课堂演示

教师、教材与交流

教师：徐国胜

——Email: guoshengxu@bupt.edu.cn

——电话: 18910970501 (wx)

——QQ: 59845049

——微信个人公众



教材

C和C++安全编码，第2版，机械工业出版社

Oday安全：软件漏洞分析技术

- 王清
- 电子工业出版社

其他网络资源

交流

- 需要大家关注公众号，期末可以取消关注
- 在云邮空间和教务网站上都有填QQ群号：1018420775
- 请学委把联系方式发给我，方便有一些问题和同学们交流



课程目标

- 介绍软件安全概念，普及软件安全意识；
- 使学生能够认识到软件开发及使用中存在的风险；
- 理解C/C++中安全编码缺陷根源、防范措施，学会初步的软件安全风险识别能力；还需要针对鸿蒙移动应用进行初步了解。
- 强调动手，使学生掌握初步的软件攻防能力，通过实验环节来实现。

理论课介绍

- 软件安全概述（含课程介绍）
- 字符串安全
- 指针安全
- 动态内存管理安全
- 整数安全
- 格式化输出
- 多线程安全
- 文件I/O安全

理论课安排同学同步演示，这个将作为平时成绩1个加分项，就是需要同学报名。

- 本学期增加鸿蒙移动应用开发的内容，作为课后自主学习内容，请大家知悉，目的就是辅助鸿蒙开源贡献。

实验课介绍

- 实验课的目的：使学生掌握初步的软件攻防能力，提高动手能力。
- 每一次实验包括若干部分实验题。一共8次实验，预计选择其中的5次作为必做部分。
- 实验部分还包括开源贡献，开源贡献分为两个部分：开放麒麟社区和开放鸿蒙社区贡献。
- 总计6次实验作为实验的全部。所占比例待定。
- 理论课每次课安排一次同学展示，具体内容考核要点讲述。

实验初步内容

- 1、缓冲区溢出
- 2、栈溢出
- 3、代码注入
- 4、漏洞挖掘与模糊测试
- 5、虚拟函数攻击
- ~~6、SEH攻击~~
- ~~7、格式化输出~~
- ~~8、SQL注入~~

+ 开源贡献

1) 鸿蒙移动应用+鸿蒙
开源贡献

2) 麒麟开源贡献

每次实验将安排1名同学，进行实验展示，暂时安排是根据实验完成情况，老师选择同学，经过沟通安排进行展示，根据展示情况，将作为平时成绩一个加分项。

考核要点

- 平时：必做（包括课堂作业、资讯）、考勤（偶尔点名）、课堂演示选作，以加分项计入平时成绩；
- 实验：一共8次实验，选择其中的5次实验，外增加开源贡献作为一次实验，包括开源麒麟社区和开源鸿蒙社区，实验成绩的6个部分，比例待定；
- 期末：期末考试卷面成绩，对应理论课，实际也会和实验有关系。

平时

平时成绩分为必做和选做两个部分：

一、必做

包括考勤，与课堂作业同步，也可能有点名；课堂作业；小资讯（每个人2条资讯，考核包括提交和阅读）

- (1) 课堂作业采用数学作业纸，一般一张纸，当堂提交；
- (2) 其余采用电子版提交。电子版提交平台，电子版作业提交需要大家按照约定时间来提交，延迟视为未提交。
- (3) 提交资讯的时间会做一个详细的分配，以免出现期末提交风暴！

二、选做（报名，加分项）

理论课堂演示，请报名同学先行做，1次课1个同学，报名完毕，和同学讨论，任务是什么，主要是伴随课堂把书上的代码穿起来演示，加分项，平时成绩。

实验演示，不需要报名。

提交要求

(1) 除非单个文档的，否则需要打包，可以是zip，也可以是rar，并且文件名需要依次包含你的作业类别、编号、班级、学号、姓名和提交时间；

(2) 小资讯是课程相关新资讯，本学期每个人都需要提交不少于2条，不同同学不可以提交相同的，提交资讯时，需要有标题，自己的评论，评论字数不限，但是务必要能够表达一个完整的意思，最后附上资讯原始的网址和能够体现这则新闻的图片一张。资讯使用word文档提交统一地址。资讯我这边使用个人公众号发布给大家看，请大家本学期关注。期末可以取消关注。**要求大家仔细阅读小资讯，并且回复：学号（可以包括姓名和自己的评论）。**

由于会有多门课程公用一个公众号，所以会在消息上显示是那个课程的，大家只需要负责回复本课程的就可以。

(3) 作业和资讯命名格式：**资讯/课堂作业/实验-01-805-2019211592-张杉-20210302.doc/docx/zip/rar/jpg/png**（类别次数编号/条目数-三位班级号-学号-姓名-提交时间.zip/rar）

(4) 开源提交另行确定。

资讯范例

包括4个部分，标题，图片，评论/描述，网址，
放到一个Word中，按照命名要求，提交到教学
云平台，描述/评论必须表达一个完整的含义。

标题：比特币交易所会重启吗？听听周小川怎么说

图片：提供图片一张

描述：当前中国政府对新技术的态度是谨慎的，
应该说是还是比较得当的。

网址：http://www.sohu.com/a/225294468_313170

关于演示

分为理论课演示和实验课演示。理论课随着理论课，实验课分享自己完成的作业即可。请大家报名参与。

(1) 理论课演示：

预计9次理论课，第一次来不及减去1次，共8次理论课需要演示，每一次课需要1名同学演示。视演示情况平时成绩最高增加10分的平时加分（平时100分的前提下）。

(2) 实验课演示：

5次实验课，参照理论课情况。

开源贡献

- 开源贡献是算一次实验，成绩纳入实验室成绩，占实验成绩比例，详细待定
- 具体安排如下：
 - 1、参与OpenHarmony开源贡献一项
 - 2、参与openKylin开源贡献一项。

内容安排

- 基本概念
 - 1 软件与软件安全
 - 2 软件安全
 - 3 漏洞
 - 4 软件风险

1.1 软件与软件安全

软件，一系列按照特定顺序组织的计算机数据和指令的**集合**。主要划分为三大类：

- **系统软件**为计算机使用提供最基本的功能
- **应用软件**是为了某种特定的用途而被开发的软件
 - 微软的Office软件
 - 数据库管理系统
- **支撑软件**是协助用户开发软件的工具性软件
 - java开发中的jdk套件

软件安全：采用工程的方法使得软件在敌对攻击的情况下仍能够继续正常工作。软件安全威胁来自于两个方面：

- **软件自身**：软件具有的漏洞和缺陷会被不法分子利用
- **外界**：黑客通过编写恶意代码并诱导用户安转运行

1.2软件安全威胁

软件开发中的不同阶段，会遇到不同的安全威胁

- **软件代码编写阶段** 敏感信息暴露等

例如：将应用的敏感信息不加密就写在发布版本中易于被黑客读取的文件中

- **软件编译阶段** 编译方式固有漏洞

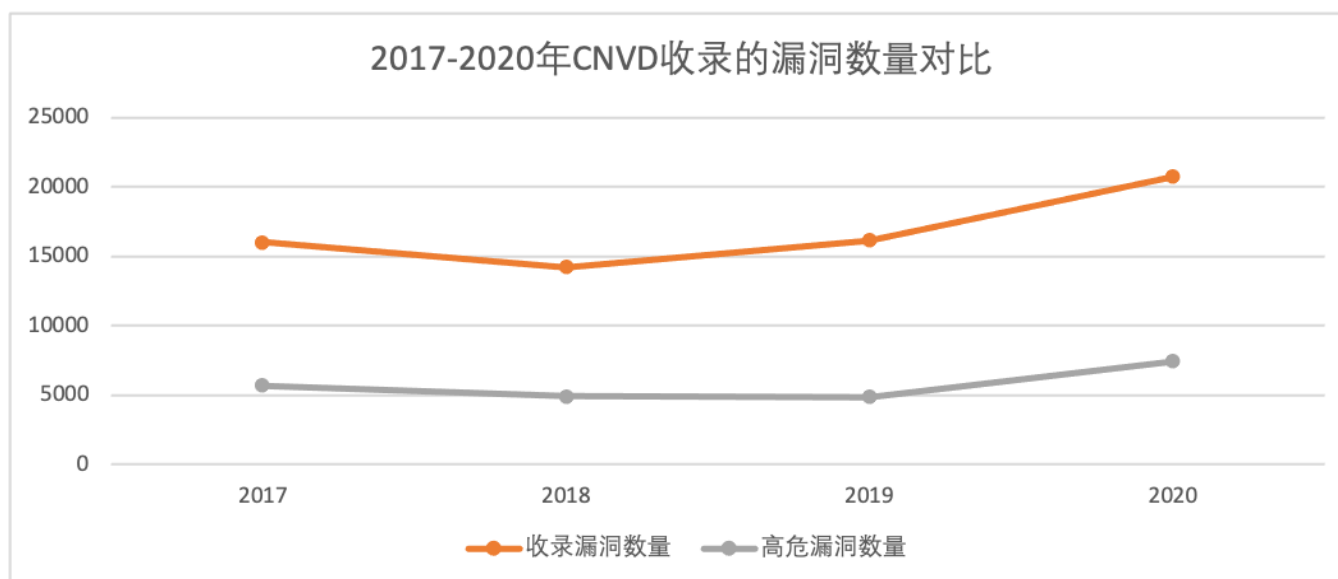
例如：Android开发中最终编译生成的smali格式文件存在固有的代码注入风险

- **软件签名发布阶段** 签名绕过威胁等

例如：在Android开发中旧版本的签名方式因为未能对所有软件文件进行校验，导致黑客可以通过修改未被校验到的文件使恶意dex文件注入

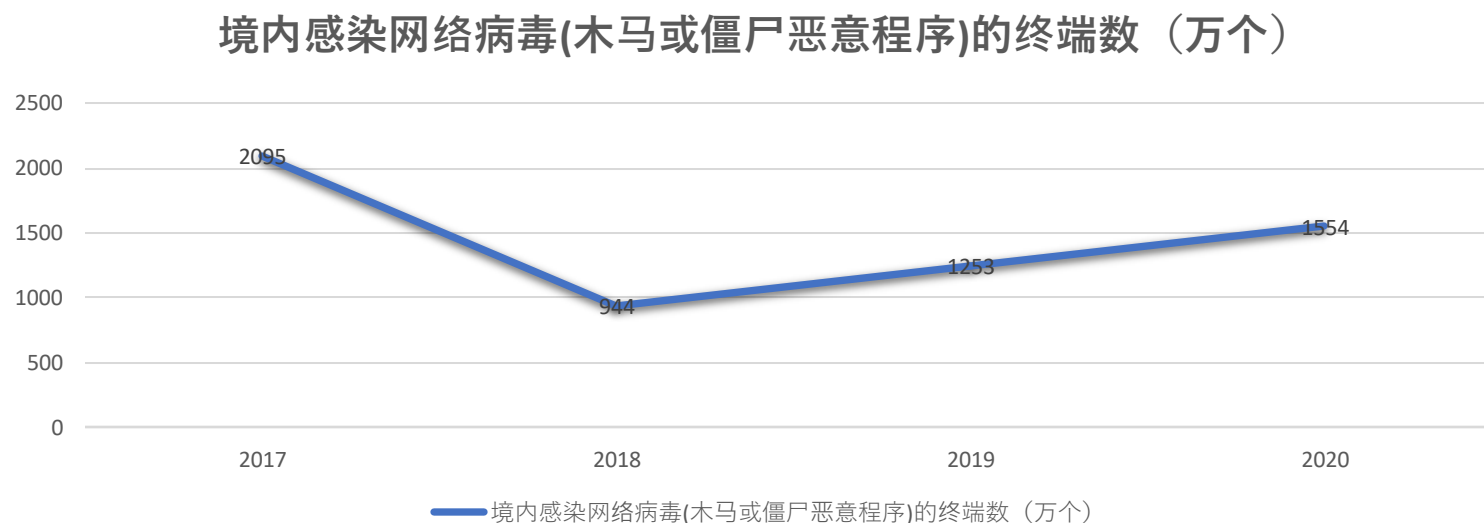
1.2 软件安全威胁

在漏洞方面，2020 年，国家信息安全漏洞共享平台（CNVD）共收录通用软硬件漏洞 **20721** 个，较 2019 年 **增长 28.2%**。



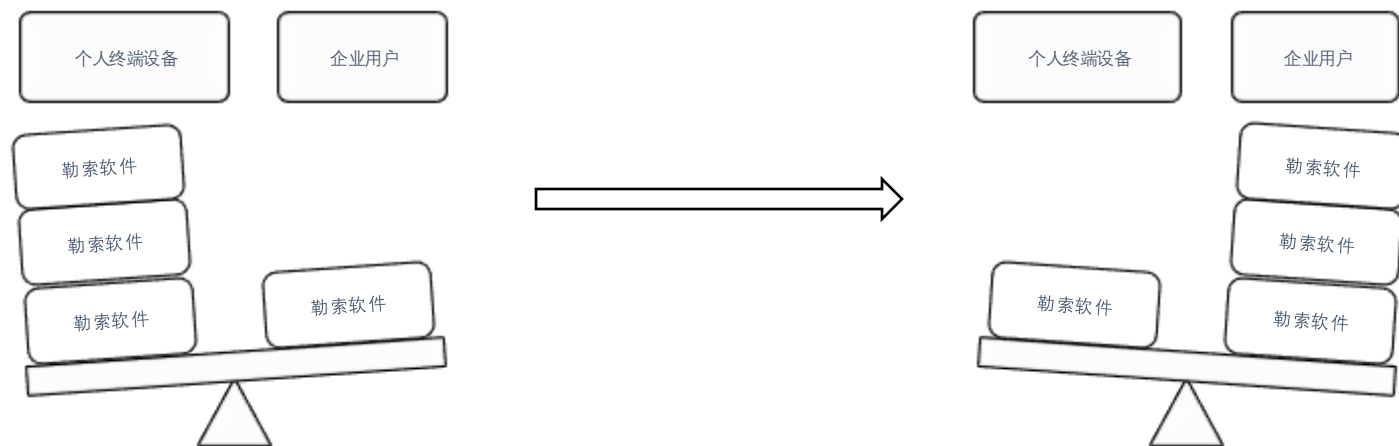
1.2软件安全威胁

2020年境内感染网络病毒(木马或僵尸恶意程序)的终端数约1554万个。



1.2 软件安全威胁

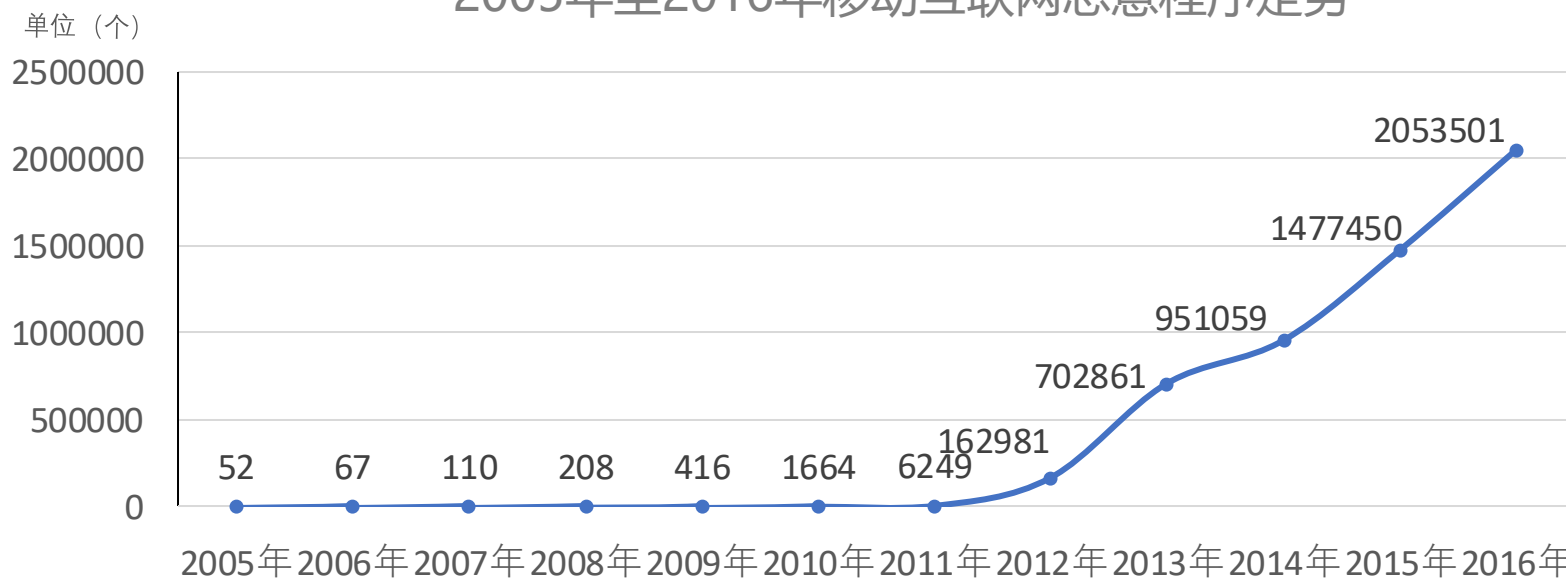
勒索软件已逐渐由针对个人终端设备延伸道企业用户



1.2软件安全威胁

互联网恶意程序下载链接近 **67 万** 条，较 2015 年增长近 **1.2 倍**，涉及的传播源域名 **22 万** 余个，IP 地址 **3 万** 余个，恶意程序传播次数达 **1.24 亿** 次。

2005年至2016年移动互联网恶意程序走势



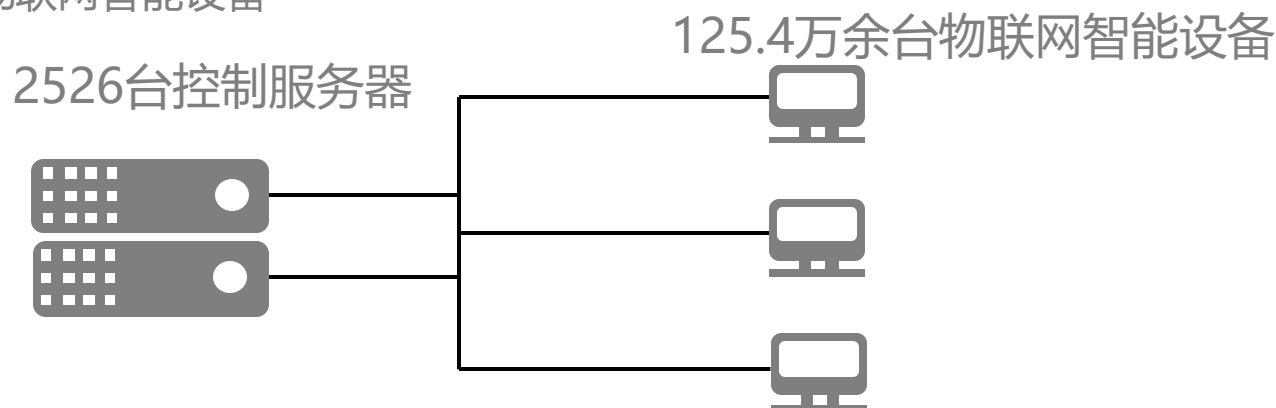
来源: CNCERT/CC

1.2软件安全威胁

数据泄露威胁日益加剧:

- 美国大选候选人希拉里的邮件泄露，直接影响到美国大选的进程
- 2016年我国免疫规划系统网络被恶意入侵，**20 万**儿童信息被窃取并在网上公开售卖

对Mirai 僵尸网络进行抽样监测：截至 2016 年年底，共发现 2526 台控制服务器控制 125.4 万余台物联网智能设备



1.3 软件的核心地位

- 软件是IT的灵魂
 - 硬件发展相对定型，硬件技术软件化
 - 业务的多样性由软件类别、功能的多样性支撑
 - 网络融合靠系统软件的支撑，比如：智能电视等

1.3 软件的核心地位

- 软件技术发生了深刻的变化
 - 软件开发者与用户之间关系发生了深刻的变化
 - 软件开发、维护模式发生了深刻的变化
 - 应用软件盈利模式正在发生深刻变化，应用软件媒体化

内容安排

- 基本概念
 - 1 软件与软件安全
 - 2 软件安全
 - 3 漏洞
 - 4 软件风险

2.1 网络空间安全

- 研究信息获取、信息存储、信息传输和信息处理领域中信息安全保障问题的一门新兴学科
- 主要围绕网络空间中电磁设备、电子信息系统、网络、运行数据、系统应用中所存在的安全问题,开展理论、方法、技术、系统、应用、管理和法制等方面的研究

- 内涵:

- 保密性: 信息不泄漏给非授权的用户、实体或者过程的特性
- 完整性: 数据未经授权不能进行改变的特性, 即信息在存储或传输过程中保持不被修改、不被破坏和丢失的特性。
- 可用性: 可被授权实体访问并按需求使用的特性, 即当需要时应能存取所需的信息。
- 真实性: 内容的真实性
- 可核查性: 对信息的传播及内容具有控制能力, 访问控制即属于可控性。
- 可靠性: 系统可靠性

- 范围
 - 网络安全：网络边界、网络协议等
 - 系统安全：信息系统软硬件等安全（软件安全：操作系统、数据库、应用软件等）
 - 内容安全：或称为信息安全、数据安全，关注数据自身保护，涉及保密、内容合法性等
 - 信息对抗：为消弱、破坏对方电子信息设备和信息的使用效能，保障己方电子信息设备和信息正常发挥效能而采取的综合技术措施

- 原由
 - 信息系统的复杂性
 - 系统软硬件缺陷，网络协议的缺陷
 - 信息系统的开放性
 - 系统开放：计算机及计算机通信系统是根据行业标准规定的接口建立起来的。
 - 标准开放：网络运行的各层协议是开放的，并且标准的制定也是开放的。
 - 业务开放：用户可以根据需要开发新的业务。
 - 人的因素

- 特点

- 攻防性：攻防技术交替改进
- 配角性：安全是属性，安全不能脱离系统、应用或业务，安全是个形容词！
- 相对性：安全性是相对的，相对主体的需求而言

- 安全业务
 - 本质是责任分解
 - 三分技术+七分管理

2.2 软件安全

- 软件安全关注计算机程序或者程序中所包含信息的完整性、机密性和可用性等

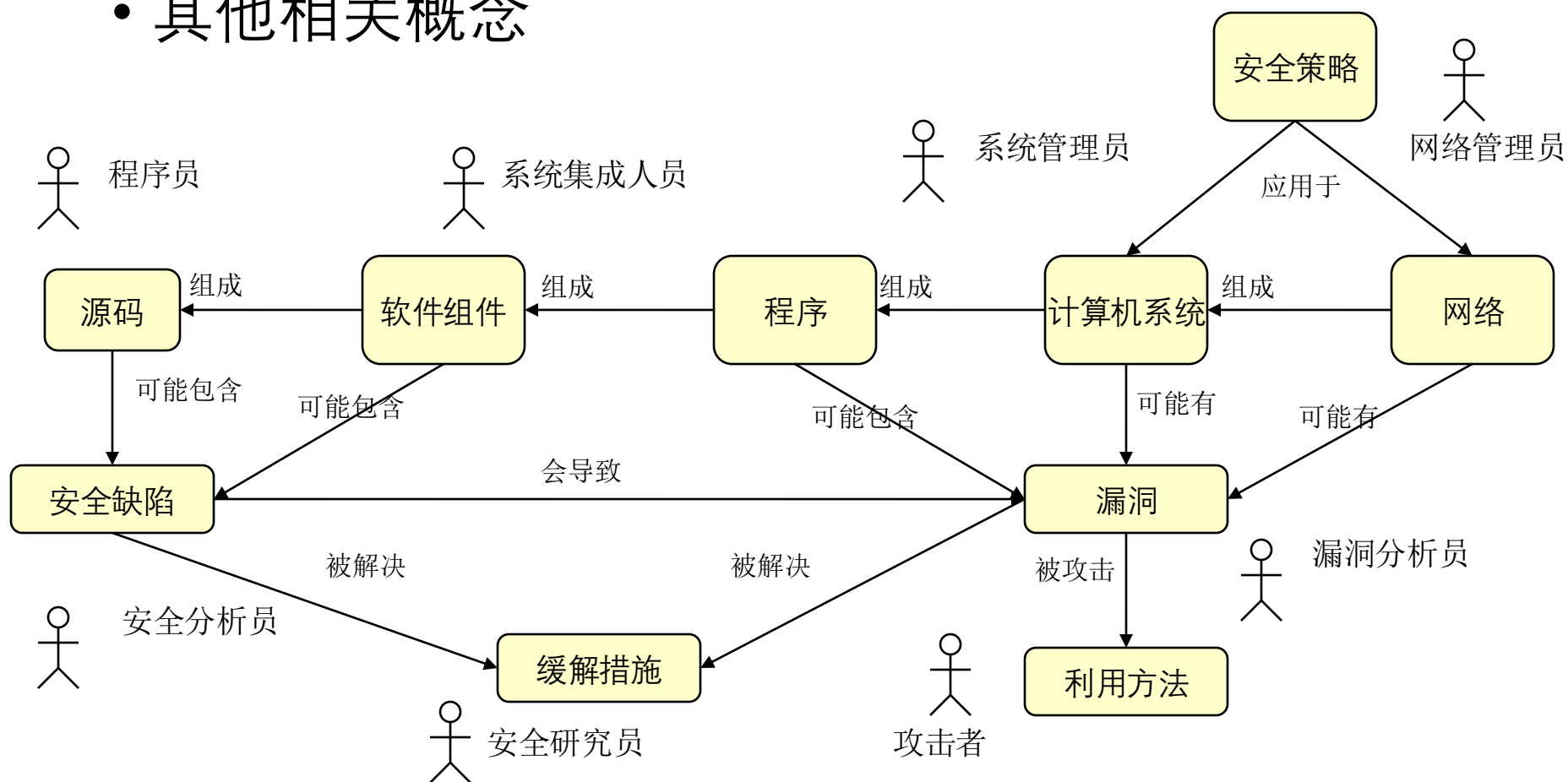
缺陷、威胁和风险

- 软件安全缺陷
 - 软件自身缺陷，设计者故意或过失而导致
 - 软件漏洞是基本形态、恶意代码则是延伸的形态
 - 软件中的客观存在

- 软件安全威胁
 - 外在的因素，过去曾经是或将来会成为攻击来源的个人、团体、组织或外部势力
 - 威胁的例子包括:黑客、内部人员、罪犯、竞争情报从业者、恐怖分子和信息战士等
 - 软件外主观存在

- 软件安全风险
 - 内在的缺陷暴露在外在威胁下的状态即为风险
 - 内在的缺陷遭遇外在威胁则形成安全事件

• 其他相关概念



- 软件安全范围



2.3 软件安全知识体系

- 软件漏洞
- 恶意代码
- 软件保护

- 软件漏洞
 - 漏洞原理与案例
 - 漏洞挖掘与利用
 - 漏洞检测与防范：安全编码

- 恶意代码
 - 恶意代码机理：传统病毒、宏病毒、木马、蠕虫等
 - 恶意代码检测：检测、消除、预防、免疫、数据备份及恢复、防范策略等

- 软件保护
 - 软件分析（破解）
 - 静态分析：控制流分析、数据流分析、数据依赖分析、别名分析、切片、抽象解析
 - 动态分析：调试、剖分、跟踪(Trace)、代码注入、HOOK、沙箱技术、反反调试

- 软件保护（反破解）
 - 防逆向分析：代码混淆、软件水印、原生代码保护、资源保护、加壳、资源及代码加密
 - 防动态调试：函数检测、数据检测、符号检测、窗口检测、特征码检测、行为检测、断点检测、功能破坏、行为占用
 - 运行环境检测、反沙箱
 - 数据校验：文件校验、内存校验
 - 代码混淆技术：静态混淆、动态混淆
 - 软件水印：静态水印、动态水印

内容安排

- 基本概念
 - 1 软件与软件安全
 - 2 软件安全
 - 3 漏洞
 - 4 软件风险

3.1 典型软件漏洞

- 缓冲区溢出
 - W32.Blaster.Worm
 - 2003年8月11日爆发
 - 在没有用户参与的情况下感染任何一台连接到互联网且未打补丁的计算机系统
 - 至少有800万个Windows系统被蠕虫感染[Lemos 04].
 - 经济损失 > \$500M

W32.Blaster.Worm

•Blaster:

- 检测计算机是否已感染
 - 将“windows auto update” = “msblast.exe” 加入注册表
HKEY_LOCAL_MACHINE\SOFTWARE\Microsoft\Windows\CurrentVersion\Run
- 启动Windows
- 产生一个随机的 IP地址
- 试图感染具有该地址的计算机
- 利用Windows 2000 或Windows XP上的DCOM RPC漏洞通过TCP 135端口发送数据。

```
1. error_status_t _RemoteActivation(  
2.     ..., WCHAR *pwszObjectName, ... ) {  
3.     *phr = GetServerPath(  
         pwszObjectName, &pwszObjectName);  
4.     ...  
5. }  
  
5. HRESULT GetServerPath(  
    WCHAR *pwszPath, WCHAR **pwszServerPath ){  
6.     WCHAR *pwszFinalPath = pwszPath;  
7.     WCHAR wszMachineName[MAX_COMPUTERNAME_LENGTH_FQDN+1];  
8.     hr = GetMachineName(pwszPath, wszMachineName);  
9.     *pwszServerPath = pwszFinalPath;  
10. }  
  
11. HRESULT GetMachineName(  
12.     WCHAR *pwszPath,  
13.     WCHAR wszMachineName[MAX_COMPUTERNAME_LENGTH_FQDN+1])  
14. {  
15.     pwszServerName = wszMachineName;  
16.     LPWSTR pwszTemp = pwszPath + 2;  
17.     while ( *pwszTemp != L'\\' )  
18.         *pwszServerName++ = *pwszTemp++;  
19.     ...  
20. }
```

SQL注入漏洞

`select * from user where username="" and password=""`

引号中的就是帐号密码输入框传入的信息。

如果我们传入的信息为 ' or '1'='1,那么这个语句就会变成

`select * from user where username="" or '1'='1' and password=""
or '1'='1'`

3.2 软件漏洞的危害

- 无法正常使用
- 引发恶性事件
- 关键数据丢失
- 秘密信息泄漏
- 被安装木马病毒

无法正常使用

对于软件安全漏洞来说，最直观的表现就是造成软件无法正常使用。

以win98为例，该操作系统曾经出现过再处理过多ping数据包时出现蓝屏的拒绝服务漏洞。

当我们用一台计算机连续不断地发送ping数据包到一台Windows 98系统的计算机时，该计算机就马上出现系统蓝屏死机的现象。

引发恶性事件

如果一个航天飞机上使用的导航控制软件出现了问题，在运行中错误地计算了飞机飞行的轨道，那么航天飞机上的宇航员就可能面临生命的危险。

计算机软件还可能被使用在注入国家安全局这样的情报部门，当一个软件发生信息泄露漏洞时，就会造成大量的国家绝密信息被泄漏，一旦敌对国家或者组织获得这些信息，那就可能造成一个国家陷入到极其可怕的境地中。

关键数据丢失

前几年，某杀毒软件曾将系统关键文件当作病毒进行了彻底删除，造成了用户的计算机系统无法正常启动，很多数据信息因此丢失，虽然引发这场危机的罪魁祸首是软件的误删除错误，而不是一个安全漏洞，但是我们能够看到软件出现问题后所造成的影响。

一些软件出现漏洞后，可能被恶意攻击者利用而进行对用户关键数据的删除，从而造成用户数据的丢失。有一些漏洞甚至会直接造成对用户数据的清除。

秘密信息泄漏

对于信息泄漏漏洞来说，用户的计算机系统等于是赤裸裸地摆在了别人的面前，里面的信息都能被其他人随意获取、查看。

对于个人是这样，对于单位、政府部门会造成更大的影响。一些敏感的有关国家军事安全、企业核心经济利益的信息，都有可能被一个软件的安全漏洞所泄漏，造成无法预知的后果。

被安装木马病毒

一个软件漏洞被发现后，最常见的利用就是恶意攻击者借此来散播木马病毒程序。我们的计算机系统中也许安装有最新的杀毒软件，最好的防火墙，但是只要计算机系统上的软件存在漏洞，那么恶意攻击者就能够在你的计算机系统上安装木马病毒程序。

3.2 软件漏洞出现的原因

- 小作坊式的软件开发
- 赶进度带来的弊端
- 被轻视的软件安全测试
- 淡薄的安全思想
- 不完善的安全维护

小作坊式的软件开发

严格地讲，任何一款计算机软件都必须依据软件工程的思想来进行设计开发。

但是，出于种种原因，很多软件的开发并没有按照软件工程的要求来实现。

有些小型公司采用了直接开发，或者边设计边开发的方法，这样只做出来的软件犹如小作坊里生产出来的产品，难免存在很多漏洞。

赶进度带来的弊端

很多大型的软件公司即使采用了软件工程思想来设计软件，但是由于事件紧迫，任务繁重，也会在一定程度上采用投机取巧或者省工省料的办法来开发软件。这个时候开发出来的软件，往往由于开发者过于疲劳或者赶进度，从而将不安全的因素带进软件，造成软件存在漏洞。

被轻视的安全测试

软件安全测试不但可以进行对软件代码的安全测试，还可以对软件成品进行安全测试。但是这样会增加成本，于是一些开发商要么不做，要么仅做最低级的测试。

这样只能保证软件能够正常使用，基本功能实现，其实，漏洞就这样被隐藏在软件内部。

淡薄的安全思想

软件安全思想主要是针对软件开发中最幸苦的编程人员来说的。由于编程人员对软件安全的认识并不一样，在做产品开发时，他们编写的代码也就对软件的安全出现了不同程度的影响。如果一个编程人员不具有一些基本的安全编程经验，他可能就会把最简单最常见的安全漏洞引入到软件内部。

不完善的安全维护

当软件出现安全漏洞时，这并不可怕。软件的开发商只需要认真检查软件出现漏洞的原因，找出修补方案就可以弥补漏洞带来的危害与损失。

可是有些开发商知道软件出现漏洞也不修补甚至欺骗用户。这种情况下，软件就会一直存在漏洞，那么使用这样软件的用户就会面临着危险

内容安排

- 基本概念
 - 1 软件与软件安全
 - 2 软件安全
 - 3 漏洞
 - 4 软件风险

GaryMcgraw软件漏洞分类

- 1 输入验证与表示
- 2 API误用
- 3 安全特征
- 4 时间与状态
- 5 错误处理
- 6 代码质量
- 7 封装
- 8 环境



输入验证与表示

- ✓ 输入验证和表示问题通常是由特殊字符、编码和数字表示所引起的，这类安全问题的发生是对输入的信任所造成的。
- ✓ 一些较严重的安全问题往往都是由于对输入的信息过度信任造成的，主要问题包括：

缓冲区溢出 (Buffer Overflow)

- 对所分配的内存块之外的内存进行写操作会覆盖数据，使程序崩溃，甚至导致执行恶意代码；

命令注入 (Command Injection)

- 在没有指定完整路径的情况下执行命令可能使得攻击者可以通过改变\$PATH或者其他环境变量来执行恶意代码；

跨站脚本 (Cross-Site Scripting)

- 向浏览器发送非法的数据会使浏览器执行恶意代码,通过XML编码进行身份认证也会导致浏览器执行非法代码；

输入验证与表示

格式化字符串 (Format String)

- 允许攻击者控制一个函数的格式化字符串，可能会引起缓冲区溢出；

HTTP应答截断 (HTTP Response Splitting)

- 在在HTTP响应头中包含未经验证的数据将可能导致缓存中毒、跨站脚本、跨用户攻击或者页面劫持攻击

非法指针值 (Illegal Pointer Value)

- 函数可能返回指向缓冲区外边的内存，接下来对该指针的操作有可能造成缓冲区溢出；

整数溢出 (Integer Overflow)

- 不考虑整数溢出会造成逻辑错误或缓冲区溢出；

日志伪造 (Log Forging)

- 写未验证数据到日记文件让攻击者伪造日记或注入恶意的内容

目录游历 (Path Traversal)

- 允许用户通过输入来控制文件系统操作使用的路径，会使攻击者有机会访问或修改被保护的系统资源；

输入验证与表示

进程控制 (Process Control)

- 装载来自于非信任源或者非信任环境的库可能会导致应用程序执行攻击者的恶意代码;

资源注入 (Resource Injection)

- 允许用户输入控制资源ID可能会使攻击者访问或修改系统保护的资源;

配置操纵 (Setting Manipulation)

- 允许外部控制系统设置有可能导致服务崩溃或应用程序进行非法操作;

SQL注入 (SQL Injection)

- 允许用户输入来构造一个动态的SQL请求, 将有可能让攻击者控制SQL语句的意思, 执行任意的SQL指令;

字符串结束错误 (String Termination Error)

- 依赖于字符串终止符有可能造成缓冲区溢出;

未检查的返回值 (Unchecked Return Value)

- 没有检查方法的返回值, 可能会导致程序进入不可预料的状态。

API误用

- ✓ API是调用者与被调用者之间的一个约定，大多数的API误用是由于调用者没有理解约定的目的所造成的。
- ✓ 举个例子，如果一个程序在调用`chroot()`后调用`chdir()`失败，那是因为它违背了约定所指定的如何在安全模式下改变动态root目录。

API误用

- API误用包括以下方面:

危险函数 (Dangerous Function)	不能被安全使用的函数不应该使用;
目录限制 (Directory Restriction)	不合理的使用系统调用chroot()可能使攻击者逃脱chroot的限制
堆检查 (Heap Inspection)	堆检查经常出现类似密码等敏感信息因为没有及时从内存中移走从而被黑客利用。所以不应该使用realloc()重新分配藏有敏感信息的缓冲区;
认证误用 (Often Misused: Authentication)	攻击者经常伪造DNS进行攻击;
异常处理误用 (Often Misused: Exception Handling)	_alloca()函数会抛出stack overflow exception, 从而会使程序当掉;
路径操纵误用 (Often Misused: Path Manipulation)	传送一个大小不够的输出缓冲区给路径操纵函数会导致缓冲区溢出;

API误用

- API误用包括以下方面:

权限管理误用（Often Misused: Privilege Management）	未能坚持最小权限原则会增大其他弱点出现的风险；
字符串操纵（Often Misused: String Manipulation）	在进行多字节和unicode字符串转换时容易发生缓冲区溢出；
未检查的返回值（Unchecked Return Value）	不考虑方法的返回值会导致程序出现意外的状况。

安全特征

- 安全特征主要指认证，访问控制，机密性，密码，权限管理等方面的内容：

不安全随机数（ Insecure Randomness ）	标准的伪随机数生成器不能抵抗加密攻击；
最小权限违规（ Least Privilege Violation ）	程序在运行到需要将权限提高的函数时将权限提高，当不需要时应立即将权限降到最低
缺少访问控制（ Missing Access Control ）	程序未能在所有可能的执行路径执行访问控制检查
口令管理（ Password Management ）	用明文存储密码或者口令为空将导致系统的不安全。在 HTTP 重定向报文发送密码会导致密码泄露
隐私违规（ Privacy Violation ）	对私有信息的不恰当处理会给用户带来损害。

时间与状态

- ✓ 分布式计算是与时间和状态有关的。也就是说，为了让多个部件之间交互，需要状态共享，而这要花费时间。
- ✓ 大多数程序员将他们的工作人格化。他们认为控制器的一个线程会按照他们所想的方式来执行整个程序。然而，现在的计算机能在多任务之间快速切换，采用多核、多CPU或者说分布式系统，两个事件甚至能在同一时间发生。
- ✓ 这样一来，在程序员所想的程序执行模式和实际发生的情况之间就会产生问题。这些问题可能涉及到线程、进程、时间和信息之间的非法交互。这些交互通过共享的状态产生：如信号量、变量、文件系统以及任何能够存储信息的东西。

时间与状态

- 包括以下方面:

死锁（ Deadlock ）	不一致的程序锁定会导致死锁；
固定会话（ Session Fixation ）	通过认证后使用同一个会话会导致攻击者劫持已认证过的会话
文件访问竞争条件 TOCTOU （ File Access Race Condition: TOCTOU ）	文件属性被验证和文件被使用之间的时间差可以引起文件使用权限提升的攻击
不安全临时文件（ Insecure Temporary File ）	创建和使用临时文件可能会导致应用程序和系统受到攻击；

错误处理

- ✓ 错误和错误处理代表了一类API。与错误处理有关的错误是很常见的。与API误用相比，和错误处理相关的安全漏洞一般是两种方式造成的：
 - ✓ 第一种是根本忘记处理错误或者只是简单的处理，并没有彻底解决
 - ✓ 第二种则是程序对可能的攻击者泄露了过多的信息或者涉及面太广没有人愿意去处理这些问题。

错误处理

- 包括以下方面:

捕获空指针异常（ Catch NullPointerException ）	捕获一个空指针异常不是一个有效的防止空指针发生的办法
空的捕获（ Empty Catch Block ）	忽视异常以及其他错误可能会导致攻击者引发意想不到的行为
过度捕获异常（ Overly Broad Catch Block ）	捕获过多的异常会使错误处理代码过度复杂，使代码更可能携带安全弱点

代码质量

- 低劣的代码质量会导致不可预测的行为。从用户的角度来看，这通常会表现为低劣的可用性。对于攻击者而言，低劣的代码使他们可以以意想不到的方式威胁系统：

双重释放（Double Free）	对同一个内存地址调用2次free()会导致缓冲区溢出
内存泄露（Memory Leak）	内存分配却没有释放，会致资源枯竭
空指针调用（Null Dereference）	程序使用一个空指针引用时，就会抛出NullPointerException
已废弃（Obsolete）	尽量不要使用已经废弃的函数；
未定义行为（Undefined Behavior）	函数行为未定义，直到其控制参数被设置为特定的值；
未初始化变量（Uninitialized Variable）	程序可能在变量没初始化之前使用它
未释放资源（Unreleased Resource）	程序有时可能不能释放系统资源；
释放后使用（Use After Free）	在内存已被释放后使用可能会导致程序崩溃；

封装

- 封装就是划定强力的分界线。在web浏览器中，这就意味着你的代码模块不能被其他代码模块滥用。在服务端，这意味着要区分校验过的数据和未经校验的数据，区分不同用户的数据，或者区分用户能看到的和不能看到的数据。

通过名字比较类	通过名字比较类可能会对导致程序将两个不同的类认为是相同的
用户间数据泄露	通过不同的会话访问同一个对象里的变量，数据可能被泄露。例如servlet以及通过共享池保存的对象
残留的debug代码	Debug代码可能会无意识的在应用程序中创建一些入口点；
系统信息泄露	系统数据和调试信息的泄露会给攻击者对系统发动攻击机会；
信任边界违规	将可信的以及不可信的数据混杂在同一个数据结构中可能会导致程序员错误地信任未验证的数据。

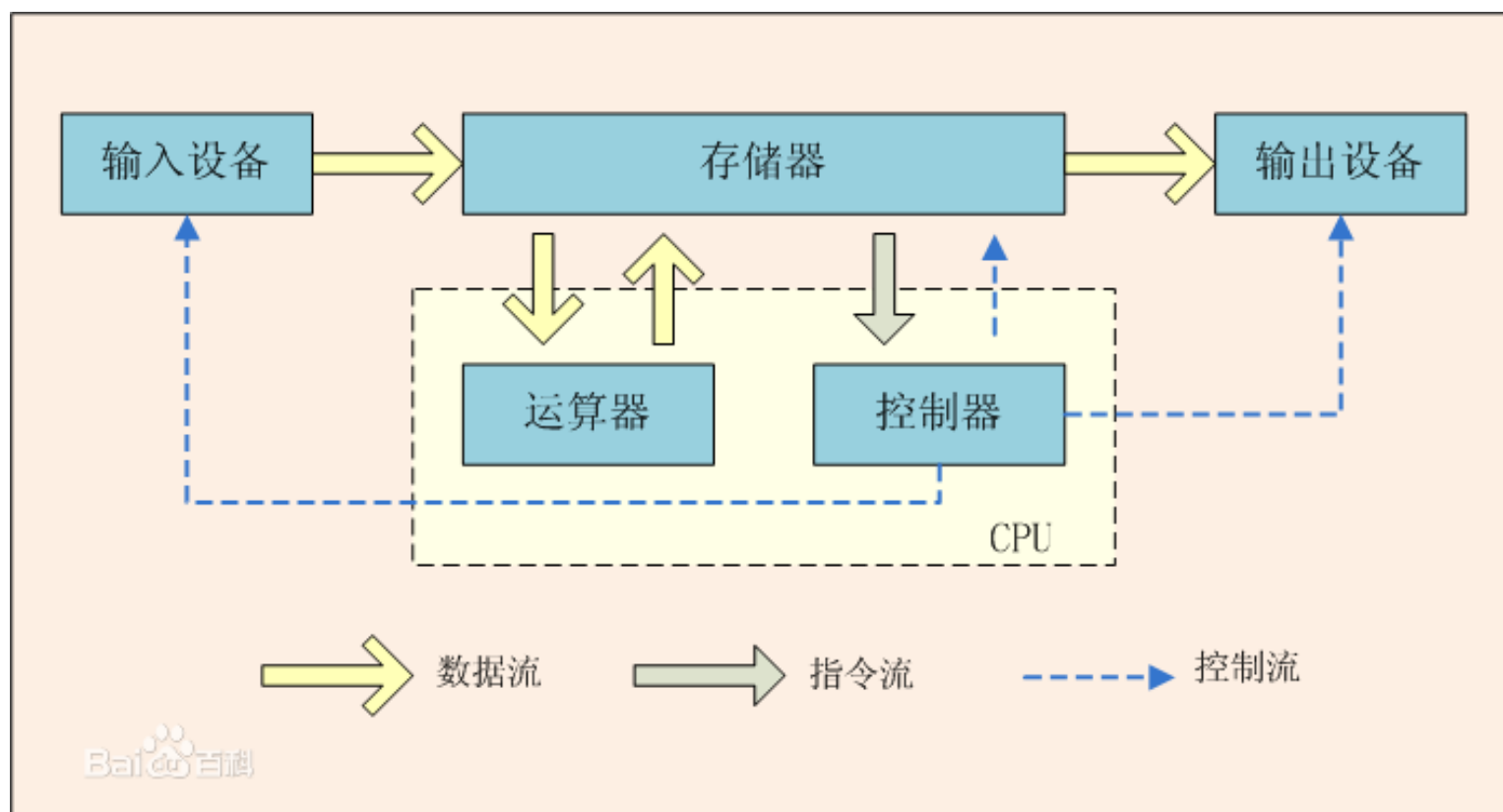
环境

- ✓环境包括的内容虽然是源代码之外的，但它们对产品的安全性仍然至关重要。因为其覆盖的内容并不与源代码直接相关，所以我们将它与其他内容分隔开来。
- ✓环境中可能存在风险例如：
 - 不安全的编译器优化(Insecure Compiler Optimization)
 - 不安全的传输(J2EE Misconfiguration: Insecure Transport)
 - 弱访问许可(J2EE Misconfiguration: Weak Access Permissions)
 - 配置文件中的密码>Password Management: Password in Configuration File)

内容安排

- 软件安全基础
 - 1 计算机引导
 - 2 Win32内存体系
 - 3 PE文件结构
 - 4 进程空间分区
 - 5 系统栈工作
 - 6 缓冲区溢出

1.1 冯·诺依曼体系



1.2 BIOS启动程序

- (1)打开计算机的电源开关时,处理器进入复位(Reset) 状态
 - 将所有内存清零, 并执行内存同位测试
 - 将段寄存器CS的内容设为FFFFH
 - 其他寄存器都清零, IP=0000H
 - 因此第一个要执行的指令是位于CS: IP中的指令, 它的物理地址为0FFFF0H, 所以将存储器的高地址分配给ROM BIOS, 作为BIOS的入口地址

- (2) 随后BIOS启动一个程序，进行主机自检
 - 确保系统的每一个部分都得到了电源支持，内存储器、主板上的其它芯片、键盘、鼠标、磁盘控制器及一些I/O端口正常可用

■ (3) 自检程序将控制权还给BIOS

■ BIOS读取BIOS设置，得到引导驱动器的顺序，依次检查

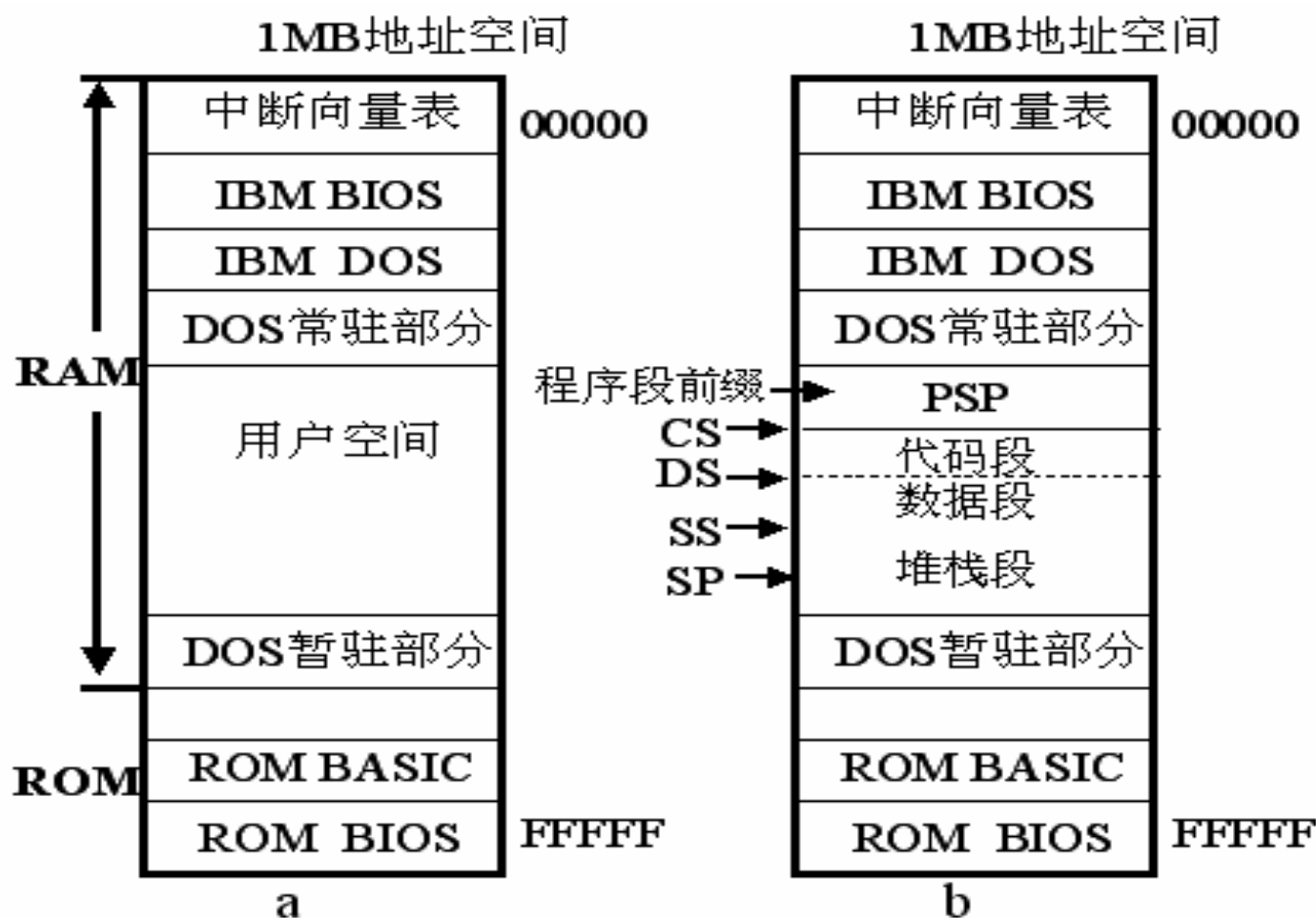
■ BIOS将所检查磁盘的第一个扇区（512B）载入内存，放在0x0000:0x7c00处，如果这个扇区的最后两个字节是“55AAH”，那么这就是一个引导扇区，磁盘也就是一块可引导盘，调用该驱动器上磁盘的引导扇区进行引导

1.3 系统加载程序

- 一旦BIOS将控制权交给操作系统之后，就可以向操作系统申请运行程序了。可执行的程序有两种：`*.com`程序及`*.exe`程序。程序在内存的配置情况如图4-1所示。
- 一个`*.com`的程序只包含一个段，段内包括指令、数据及堆栈，这种程序小而精，适合用于小的应用程序或常驻程序。汇编语言绝大部分的程序均使用`*.exe`的格式。
- 这里`*.com`程序被加载运行时有下列的特点：

- ①. 一个*.com程序占用一个段（64KB）的运行空间，PSP（Program Segment Prefix）、指令、数据及堆栈都安排在这个段内。段寄存器CS、DS、ES及SS均指向这一段的起始地址。
- ②. 段内安排以程序段前缀PSP在前，指令及数据居中，堆栈则安排在段的高地址部分。PSP占用256个字节的空間，因此第一个指令的偏移地址必须从0100H开始，IP寄存器的初值即是0100H。

- ③ . PSP及堆栈都由系统配置，系统会在堆栈顶端存入两个字节，其内容为零值，SP就指到这个零值的数据项，这时SP的内容为OFFFEH。



PC-DOS的内存配置 (a为DOS启动后, b为用户程序装入后)

1.4 Windows系统启动过程

- 预启动：通过BIOS进行自检
- 启动
- 装载内核
- 内核初始化
- 用户登录

①预启动

按电源开机

CPU初始化

POST加电自检

检测显卡

检测CPU

检测所有内存

检测标准设备

检测即插即用设备

显示系统配置表

BIOS

系统BIOS

更新ESCD

系统BIOS最后按

指定驱动器启动

软驱

光驱

硬盘
引导区
NTLDR

①系统BIOS读MBR（主引导记录）检查HD分区表

②系统BIOS调NTLDR（操作系统加载器）进内存

②启动

CPU初始化
（实模式转为
32位保护模式）读取
BOOT.INI

内存

选择希望启动
的操作系统内存中的
NTLDRWindows
2000 / XP非Windows
2000 / XPNTLDR会
继续引导
进行以下过程NTLDR则会读取
系统引导扇区副本
BOOTSECT.DOS转入启动
相应系统

③装载内核

引导过程装载

Windows2000/XP内核
(NTOSKRNL.EXE)

硬件抽象层(HAL)

④初始化内核

Windows2000/XP
内核完成初始化Windows2000/XP
内核开始装载并初始化
设备驱动程序

启动WIN32子系统

启动
WINDOWS2000/XP
服务

⑤用户登录

开始登录进程

WIN32子系统
启动

WINLOGON.EXE

LOCAL
SECURITY
AUTHORITY
(LSASS.EXE)
显示登录对话框继续配置
网络设备和用户环境

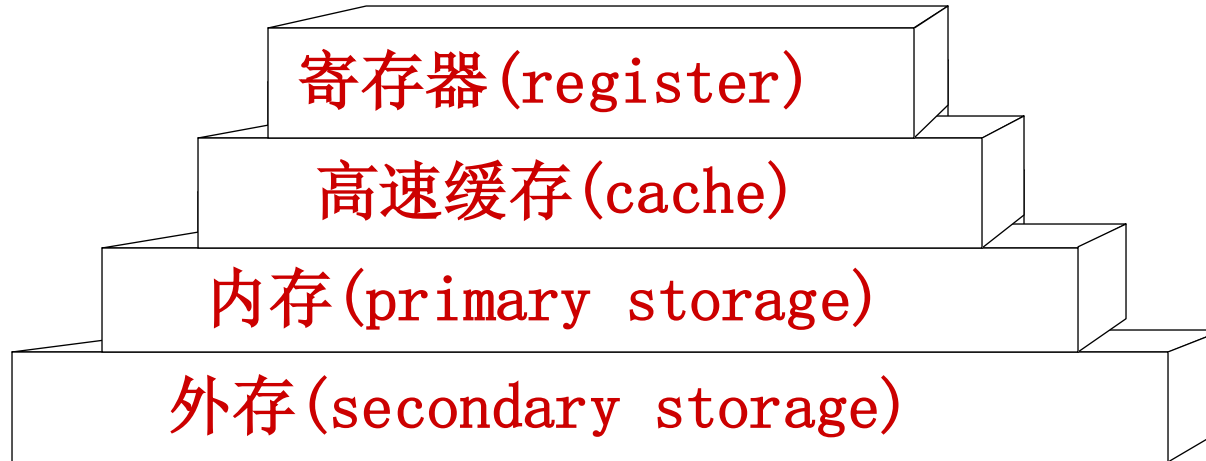
个性化桌面

86

内容安排

- 软件安全基础
 - 1 计算机引导
 - 2 Win32内存体系
 - 3 PE文件结构
 - 4 进程空间分区
 - 5 系统栈工作
 - 6 缓冲区溢出

2.1 存储体系



- 高速缓存：
 - Data Cache
 - TLB(Translation Lookaside Buffer)
- 内存： DRAM, SDRAM等；
- 外存： 软盘、硬盘、光盘、磁带等；

寄存器

Intel x86的寄存器可以分为下述的几类：

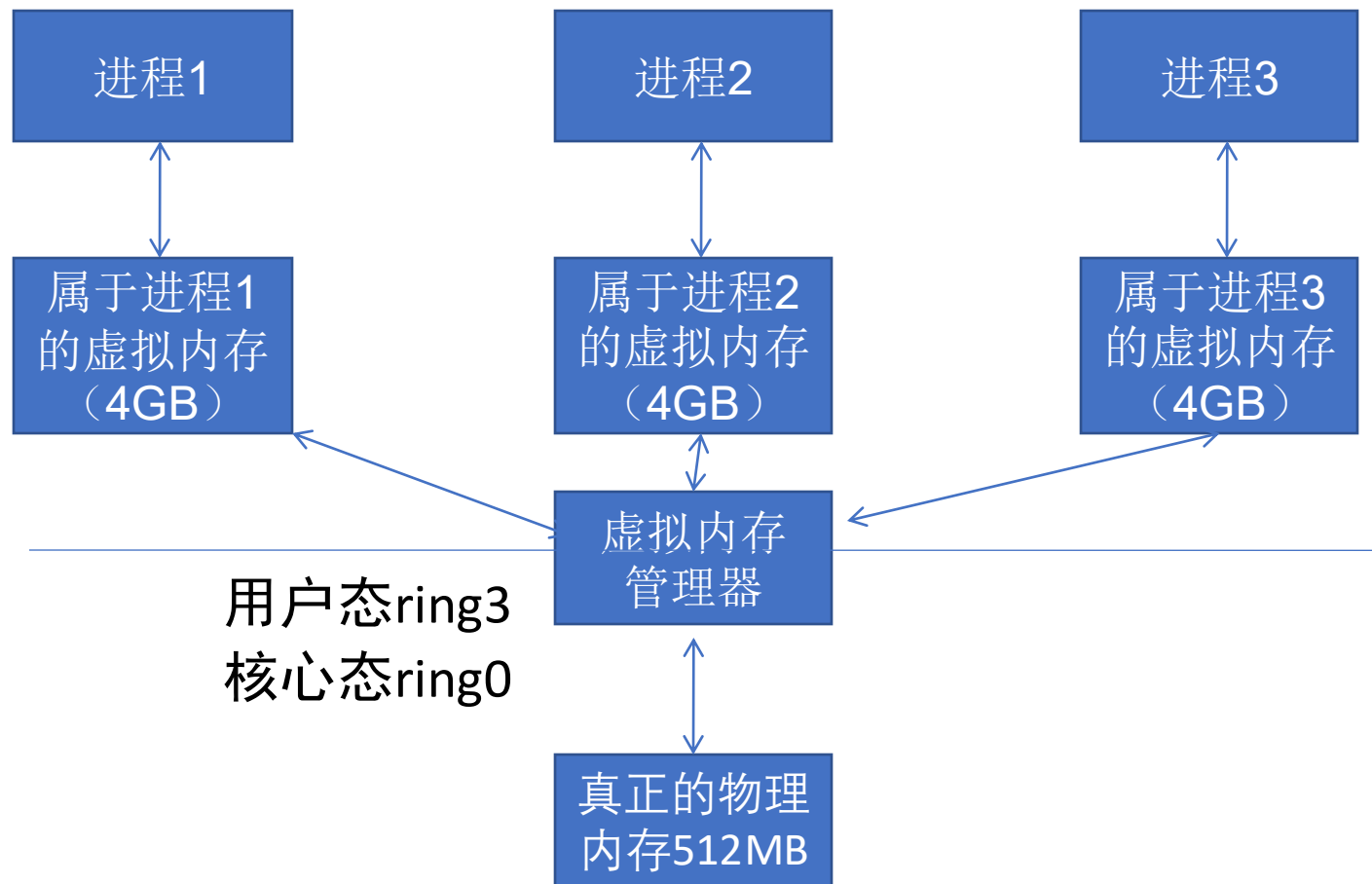
- 通用寄存器
 - 32位的通用寄存器有*EAX*、*EBX*、*ECX*、*EDX*、*ESP*、*EBP*、*ESI*和*EDI*，它们的使用方法不总是相同的。一些指令赋予它们特殊的功能。
- 段寄存器
 - 段寄存器被用于指向进程地址空间不同的段。
 - *CS*指向一个代码段的开始；
 - *SS*是一个堆栈段；
 - *DS*、*ES*、*FS*、*GS*和各种其他数据段，例如存储静态数据的段。
- 程序流控制寄存器
- 其他寄存器

寄存器

- 在通用寄存器里面有很多寄存器虽然他们的功能和使用没有任何的区别，但是在长期的编程和使用中，在程序员习惯中已经默认的给每个寄存器赋上了特殊的含义，比如：
 - EAX一般用来做返回值
 - ECX用于记数
 - EIP：扩展指令指针。在调用一个函数时，这个指针被存储在堆栈中，用于后面的使用。在函数返回时，这个被存储的地址被用于决定下一个将被执行的指令的地址。
 - ESP：扩展堆栈指针。这个寄存器指向堆栈的当前位置，并允许通过使用push和pop操作或者直接的指针操作来对堆栈中的内容进行添加和移除。
 - EBP：扩展基指针。主要用与存放在进入call以后的ESP的值，便于退出的时候回复ESP的值，达到堆栈平衡的目的。

2.2 虚拟内存

- Windows的内存可以被分为两个层面：物理内存和虚拟内存。其中，物理内存比较复杂，需要进入Windows内核级别ring0才能看到。通常，在用户模式下，我们用调试器看到的地址都是虚拟内存。
- Windows让所有的进程都“相信”自己拥有独立的4GB内存空间。但是我们计算机中那跟实际的内存条可能只有512MB,怎么能为所有进程都分配4GB的内存呢？这一切都是通过虚拟内存管理器的映射做到的。



- 虽然每个进程都“相信”自己拥有4GB的空间，但实际上它们运行时真正能用到的空间根本没有那么多。
- 内存管理器只是分给进程一片“假地址”，或者说“虚拟地址”，它们对进程来说只是一笔“无形的数字财富”；
- 当需要实际的内存操作时，内存管理器才会把“虚拟地址”和“物理地址”联系起来。

2.3 内存管理与银行的类比

我们将银行与内存管理机制进行一下类比来帮助大家理解

内存管理	银行类比
进程	储户
内存管理器	银行
物理内存	钞票
虚拟内存	存款
进程可能拥有大片内存，但使用往往很少	储户拥有大笔存款，但实际生活中的开销并没多少

我们将银行与内存管理机制进行一下类比来帮助大家理解

内存管理	银行类比
进程不使用虚拟内存时，这些内存只是些地址，是虚拟存在的，是一笔无形的数字财富。	用户不使用储蓄时，储蓄也只是些数字，是无形的数字财富。
进程使用内存时，内存管理器会为此虚拟地址映射实际的物理地址，虚拟内存地址和最终被映射到的物理内存地址之间没有什么必然联系	储户需要钱时，银行才会兑换一定的现金给储户，但物理钞票的号码与储户心目中的数字存款之间可能并没有任何联系。

我们将银行与内存管理机制进行一下类比来帮助大家理解

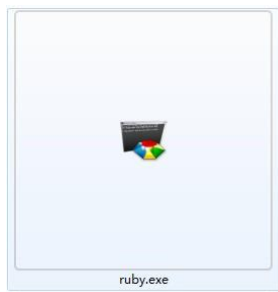
内存管理	银行类比
操作系统的实际物理内存空间可以远远小于进程的虚拟内存空间之和，仍能正常调度	银行的现金准备可以远远小于所有储户的储蓄额总和，仍能正常运转
很少出现所有程序要申请出全部物理内存	很少会出现所有储户同时要取出全部存款的现象
物理内存可以远小于虚拟内存之和	社会上实际流通的钞票可以远远小于社会的财富总额

内容安排

- 软件安全基础
 - 1 计算机引导
 - 2 Win32内存体系
 - 3 PE文件结构
 - 4 进程空间分区
 - 5 系统栈工作
 - 6 缓冲区溢出

3.1 PE文件格式简介

- PE(Portable Executable)是Win32平台下可执行文件遵守的数据格式。常见的可执行文件(如“*.exe”文件和“*.dll”文件)都是典型的PE文件。



- 一个可执行文件不光包含了二进制的机器代码，还会自带其他信息，如字符串、菜单、图标、位图、字体等。

- PE文件格式规定了所有的这些信息在可执行文件中如何组织。在程序被执行时，操作系统会按照PE文件格式的约定去相应的地方准确地定位各种类型的资源，并分别装入内存的不同区域。
- PE文件格式把可执行文件分成若干个数据节(section)，不同的资源被存放在不同的节中。一个典型的PE文件包含的节如下：

- .text 由编译器产生，存放着二进制的机器代码，也是我们反汇编和调试的对象。
- .data 初始化的数据块，如宏定义、全局变量、静态变量等。
- .idata 可执行文件所使用的动态链接库等外来函数与文件的信息。
- .rsrc 存放程序的资源，如图标、菜单等。
- 除此之外，还可能出现的节包括“.reloc”、“.edata”、“.tls”、“.rdata”等。



PE文件简单构成

3.2 PE文件与虚拟内存之间的映射

- 静态反汇编工具看到的PE文件中某条指令的位置是相对于磁盘文件而言的，即所谓的文件偏移，我们可能还需要知道这条指令在内存中所处的位置，即虚拟内存的位置
- 反之，在调试时看到的某条指令的地址是虚拟内存地址，我们也经常需要回到PE文件中找到这条指令对应的机器码

我们首先要弄清楚几个概念

- 文件偏移地址(File Offset)

- 数据在PE文件中的地址叫做文件偏移地址。这是文件在磁盘上存放时相对于文件开头的偏移。

- 装载基址(Image Base)

- PE装入内存时的基地址。默认情况下, EXE文件在内存中的基地址是0x00400000, DLL文件是0x10000000。这些位置可以通过修改编译选项更改。

我们首先要弄清楚几个概念

- 虚拟内存地址(Virtual Address,VA)

- PE文件中的指令被装入内存后的地址。

- 相对虚拟地址(Relative Virtual Address,RVA)

- 相对虚拟地址是内存地址相对于映射基址的偏移量。
- 虚拟内存地址、映射基址、相对虚拟内存地址三者有如下关系

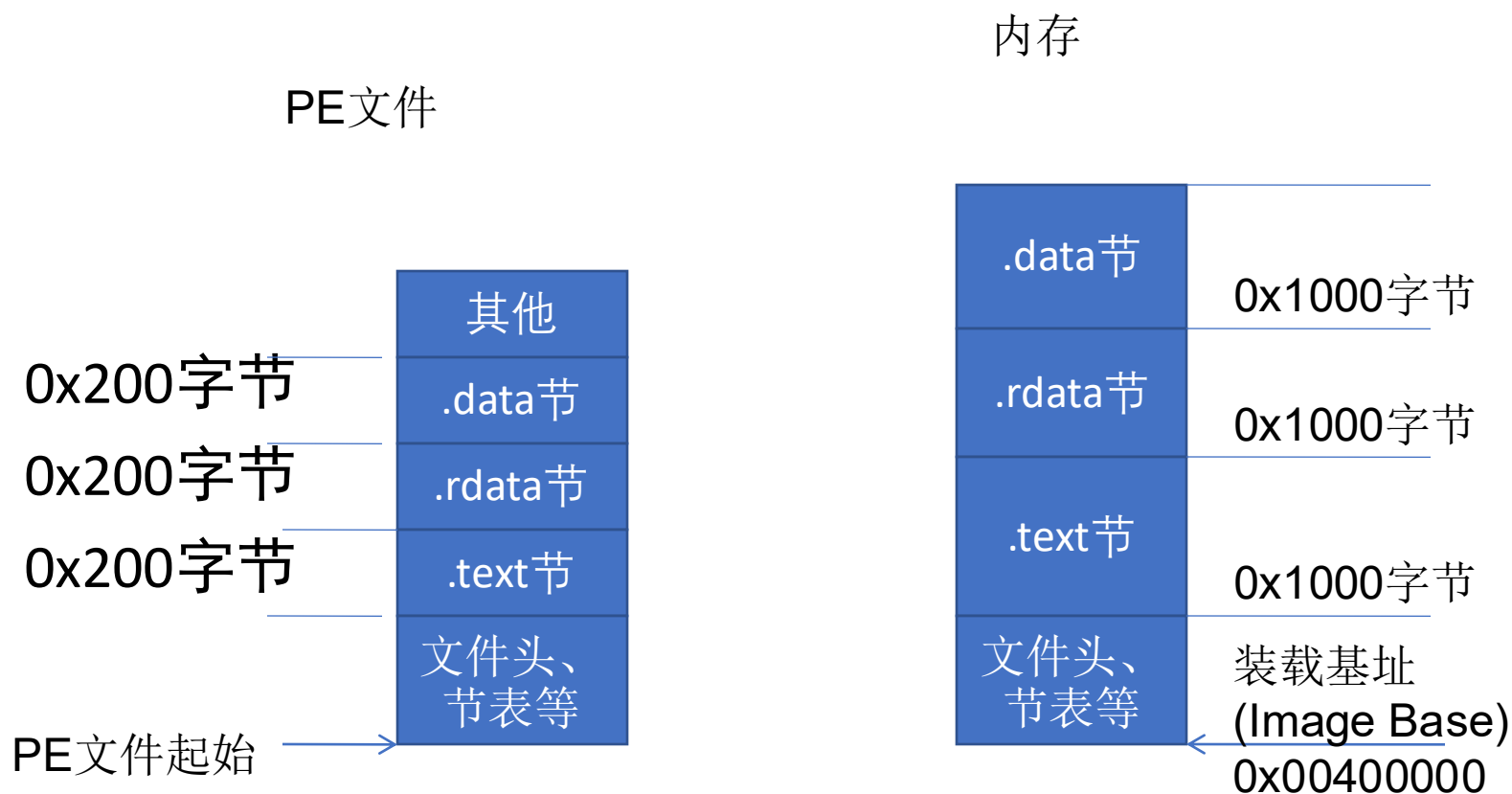
$$VA = \text{Image Base} + RVA$$

文件偏移地址在与他们计算时还需要考虑存放方式的不同。

- PE文件中的数据按照磁盘数据标准存放，以0x200字节为基本单位进行组织。当一个数据节不足0x200字节时，不足的地方将被0x00填充；当一个数据节超过0x200字节时，下一个0x200块将分配给这个节使用。因此PE数据节的大小永远是0x200的整数倍。

文件偏移地址在与他们计算时还需要考虑存放方式的不同。

- 当代码装入内存后，将按照内存数据标准存放，并以0X1000字节为基本单位进行组织。类似的，不足将被补全，若超出将分配下一个0x1000为其所用。因此，内存中的节总是0x1000的整数倍。



PE文件与虚拟内存的映射关系

节(section)	相对虚拟偏移量(RVA)	文件偏移量
.text	0x00001000	0x0400
.rdata	0x00007000	0x6200
.data	0x00009000	0x7400
.rsrc	0x0002D000	0x7800

我们把这种由存储单位差异引起的节基址差称做**节偏移**，在上图例中：

.text节偏移 = $0x1000 - 0x400 = 0xc00$

.rdata节偏移 = $0x7000 - 0x6200 = 0xE00$

.data节偏移 = $0x9000 - 0x7400 = 0x1c00$

.rsrc节偏移 = $0x2D000 - 0x7800 = 0x25800$

文件偏移地址 = 虚拟内存地址(VA) – 装载基址(Image Base)-节偏移
= RVA – 节偏移

以上表为例，如果在调试时遇到虚拟内存中0x00404141处的一条指令，那么要换算出这条指令在文件中的偏移量，有：

文件偏移量 = $0x00404141 - 0x00400000 - (0x1000 - 0x400) = 0x3541$

内容安排

- 软件安全基础
 - 1 计算机引导
 - 2 Win32内存体系
 - 3 PE文件结构
 - 4 进程空间分区
 - 5 系统栈工作
 - 6 缓冲区溢出

4.1 进程空间的功能分区

根据不同的操作系统，一个进程可能被分配到不同的内存区域去执行。但是不管什么样的操作系统、什么样的计算机架构，进程使用的内存都可以按照功能大致分成以下4个部分。

内存的不同用途

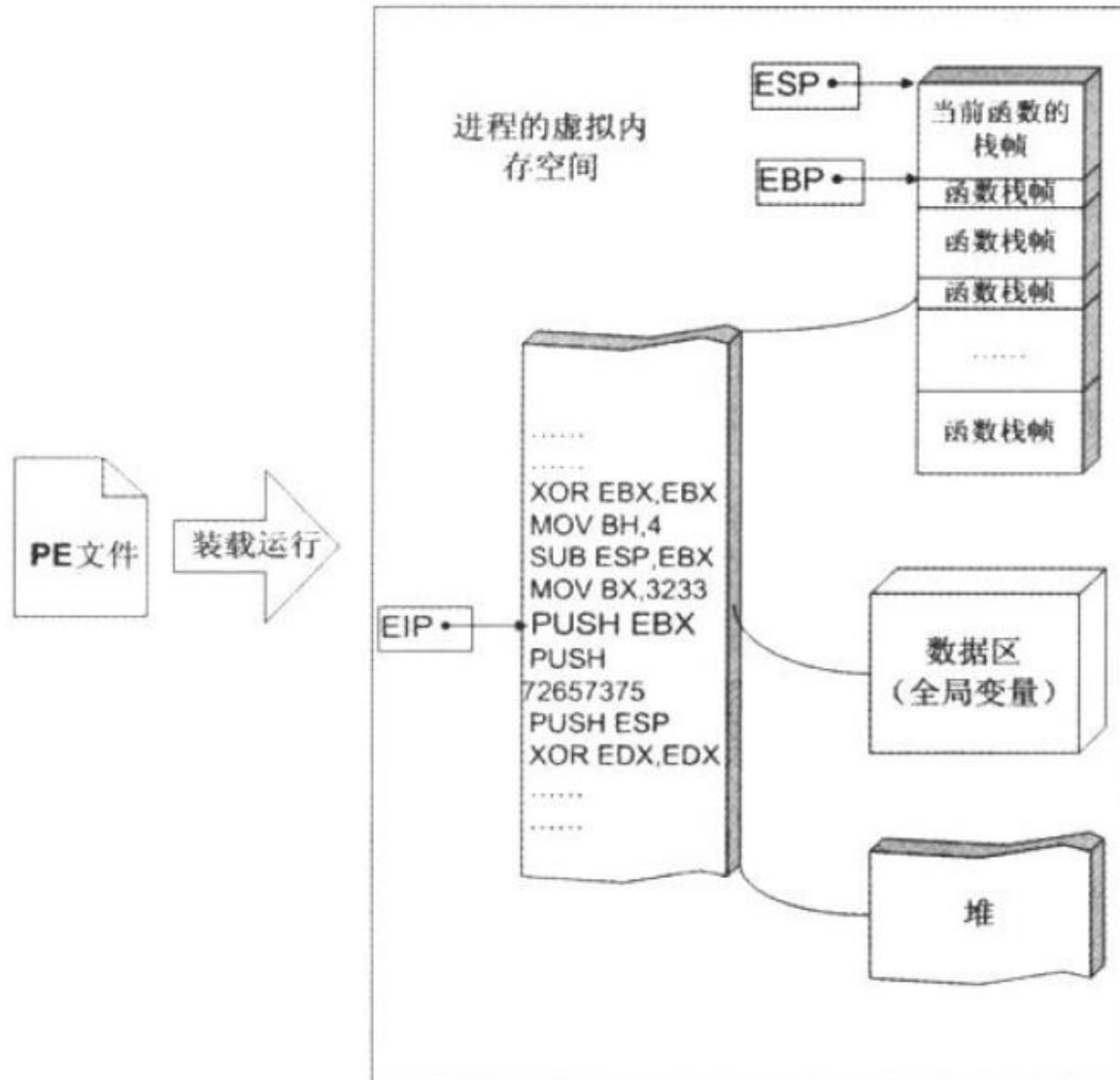
名称	用途
代码区	这个区域存储着被装入执行的二进制机器代码，处理器会到这个区域取指并执行。
数据区	用于存储全局变量等。
堆区	进程可以在堆区动态地请求一定大小的内存，并在用完之后归还给堆区。动态分配和回收是堆区的特点。
栈区	用于动态地存储函数之间的调用关系，以保证被调用函数在返回时恢复到父函数中继续执行。

内存的4个部分和相关用途

如果把计算机看成一个有条不紊的工厂，我们可以得到如下类比。

名称	类比
CPU	完成工作的工人
数据区、堆区、栈区等	存放原料、半成品、成品等各东西的场所。
存在代码区的指令	告诉CPU要做什么，怎么做，到哪里去领原料，用什么工具来做，做完以后把成品放到哪个货舱去。
栈区	栈除了扮演存放原料、半成品的仓库之外，它还是车间调度主任的办公室。

内存与工厂的类比关系



4.2 栈与系统栈

程序中所使用的缓冲区可以是堆区、栈区和存放静态变量的数据区。缓冲区溢出的利用方法和缓冲区到底属于上面哪个内存区域密不可分，本课主要介绍在系统栈发生溢出的情形。

从计算机科学的角度来看，栈指的是一种数据结构，是一种先进后出的数据表。

栈的最常见操作有两种：压栈(PUSH)、弹栈(POP)；用于标识栈的属性也有两个：栈顶(TOP)、栈底(BASE)。

为了便于理解，我们把栈和扑克牌进行一下类比

操作名称	类比效果
PUSH	为栈增加一个元素的操作叫做PUSH，这相当于在这摞扑克牌的最上面再放上一张牌。
POP	从栈中取出一个元素的操作叫做POP,相当于从这摞扑克牌取出最上面的一张。

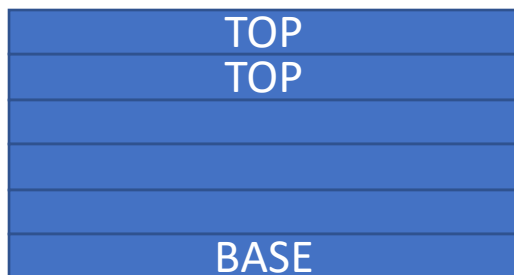
栈操作与扑克的类比

假如我们把栈想象成一摞扑克牌：

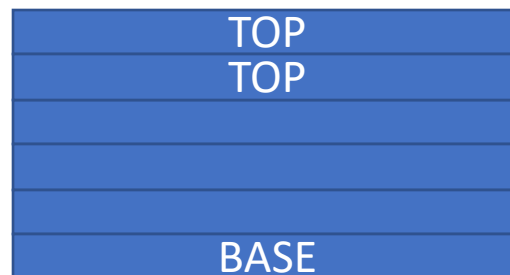
位置名称	类比效果
TOP	标识栈顶的位置，并且是动态变化的。每做一次PUSH操作，它都会自增1；相反，每做一次POP操作，它会自减1。栈顶元素相当于扑克牌最上面一张，只有这张牌的花色是当前可以看到的。
BASE	标识栈底的位置，它记录着扑克牌最下面一张的位置。BASE用于防止栈空后继续弹栈。一般情况下它的值是不会改变的。

栈位置属性与扑克牌的类比

假如我们把栈想象成一摞扑克牌：



PUSH操作



POP操作

栈位置属性与扑克牌的类比

内存的栈区实际上指的就是系统栈。系统栈由系统自动维护，它用于实现高级语言中函数的调用。对于类似C语言这样的高级语言，系统栈的PUSH/POP等堆栈平衡细节是透明的。

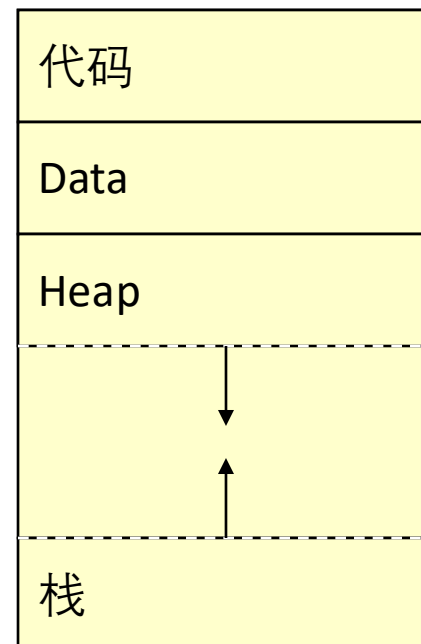
一般说来，只有在使用汇编语言开发程序的时候，才需要和它直接打交道。

内容安排

- 软件安全基础
 - 1 计算机引导
 - 2 Win32内存体系
 - 3 PE文件结构
 - 4 进程空间分区
 - 5 系统栈工作
 - 6 缓冲区溢出

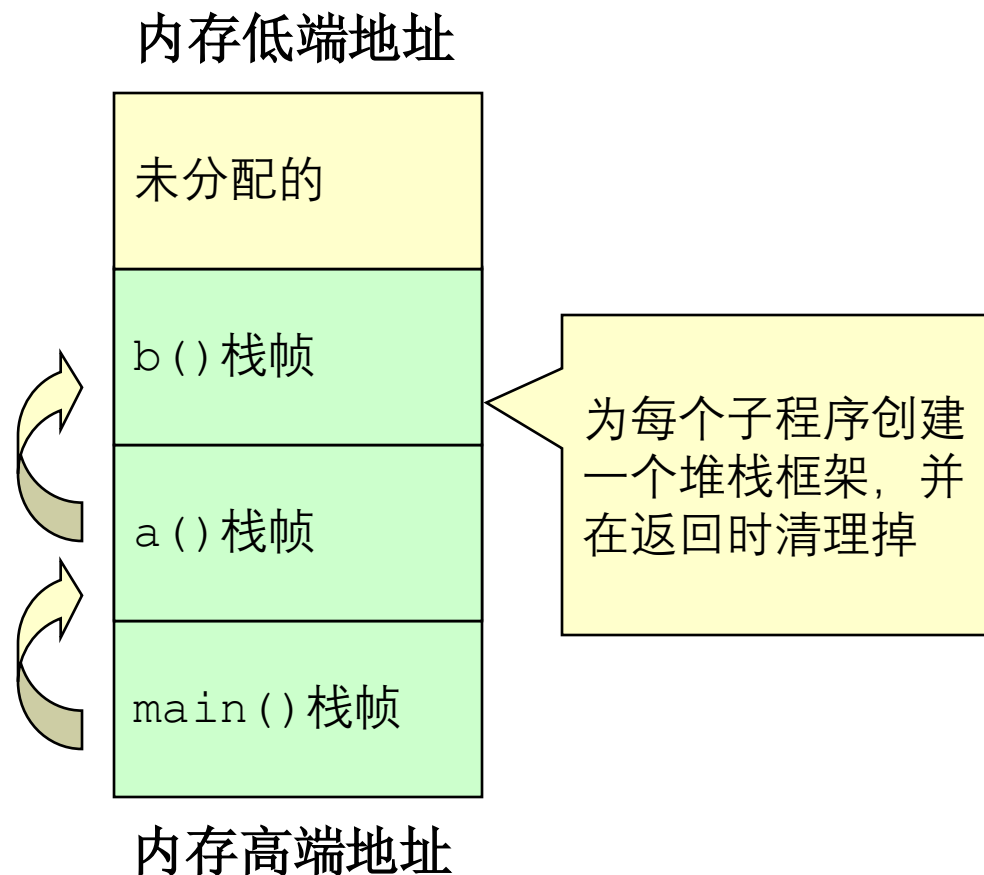
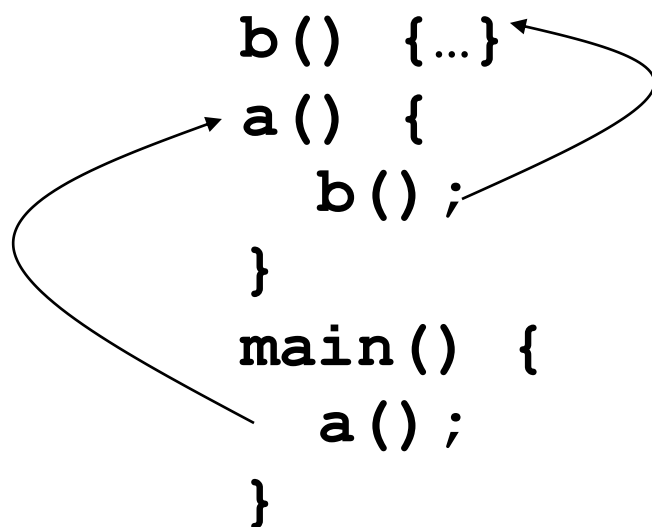
5.1 程序栈

- 栈通过存储下列内容来追踪程序的执行和状态。
 - 调用函数的返回地址
 - 函数参数
 - 局部 (临时) 变量
- 在下列情况下栈需要被修改
 - 在函数调用期间
 - 函数初始化期间
 - 从子例程返回时



程序栈

- 堆栈支持嵌套调用
- 帧指由函数调用引发的压入栈的数据。



程序栈

- 栈用于存储
 - 子例程的实际参数
 - 调用函数的返回地址
 - 局部（自动）变量
- 当前帧的地址被存储到帧或者基址寄存器中(英特尔架构中的**EBP**)
- 帧指针在栈中是一个定点的引用。
- 下列情况出现时，栈要被修改
 - 子例程调用
 - 子例程初始化
 - 从子例程返回

子例程调用

```
push 2  
push 4  
call function (411A29h)
```

把第2个参数压入栈

把第1个参数压入栈

把返回地址压入栈并跳到那个地址

• **function(4, 2);**

EIP = 00411A80 ESP = 0012FE0C EBP = 0012FEDC

EIP: 扩展指令指针 **ESP:** 扩展栈指针 **EBP:** 扩展基指针

子例程初始化

• **void function(int arg1, int arg2) {**

push ebp

存储帧指针

mov ebp, esp

子例程的帧指针被设置为当前栈指针

sub esp, 44h

为局部变量分配

EIP = 00411A29 ESP = 0012FD40 EBP = 0012FE00

EIP:扩展指令指针

ESP:扩展栈指针

EBP:扩展基指针

子例程初始化

- **function(4, 2);**

```
push 2
```

```
push 4
```

```
call function (411230h)
```

```
add esp, 8
```

存储栈指针

EIP = 00411A8A ESP = 0012FE10 EBP = 0012FEDC

EIP:扩展指令指针

ESP:扩展栈指针

EBP:扩展基指针

Subroutine Return

- `return();`

```
mov esp, ebp
```

存储栈指针

```
pop ebp
```

存储帧指针

```
ret
```

将返回地址从栈弹出，并把控制移交给那个位置

EIP = 00411A87 **ESP = 0012FE08** **EBP = 0012FEDC**

EIP:扩展指令指针

ESP:扩展栈指针

EBP:扩展基指针

程序案例

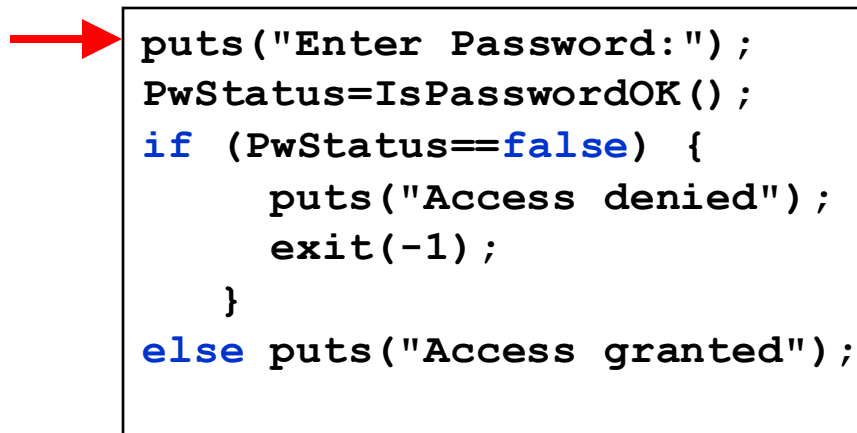
```
bool IsPasswordOK(void) {
    char Password[12]; // Memory 存储pwd
    gets(Password);    // Get input from keyboard
    if (!strcmp(Password, "goodpass")) return(true); //
    Password Good
    else return(false); // Password Invalid
}

void main(void) {
    bool PwStatus;           // Password Status
    puts("Enter Password:"); // Print
    PwStatus=IsPasswordOK(); // Get & Check Password
    if (PwStatus == false) {
        puts("Access denied"); // Print
        exit(-1);              // Terminate Program
    }
    else puts("Access granted");// Print
}
```


执行IsPasswordOK() 函数之前，栈中的信息

EIP

代码

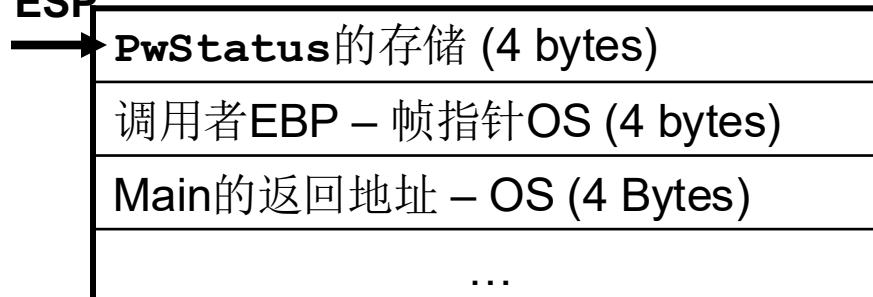


A red arrow points from the label 'EIP' to the first line of code in a box. The code is as follows:

```
puts("Enter Password:");  
PwStatus=IsPasswordOK();  
if (PwStatus==false) {  
    puts("Access denied");  
    exit(-1);  
}  
else puts("Access granted");
```

栈

ESP



IsPasswordOK () 执行时栈中的信息

代码

EIP →

```
puts("Enter Password:");
PwStatus=IsPasswordOK();
if (PwStatus==false) {
    puts("Access denied");
    exit(-1);
}
else puts("Access granted");
```

```
bool IsPasswordOK(void) {
    char Password[12];

    gets(Password);
    if (!strcmp(Password, "goodpass"))
        return(true);
    else return(false);
}
```

IsPasswordOK栈帧

ESP →	存储Password (12 Bytes)
	调用者EBP – 帧指针main (4 bytes)
EBP →	调用者的返回地址– main (4 Bytes)
	存储PwStatus (4 bytes)
	调用者EBP – 帧指针OS (4 bytes)
	Main的返回地址– OS (4 Bytes)
	...

注意: IsPasswordOK(void) 函数调用导致栈增长和收缩

IsPasswordOK () 调用后栈中的信息

代码

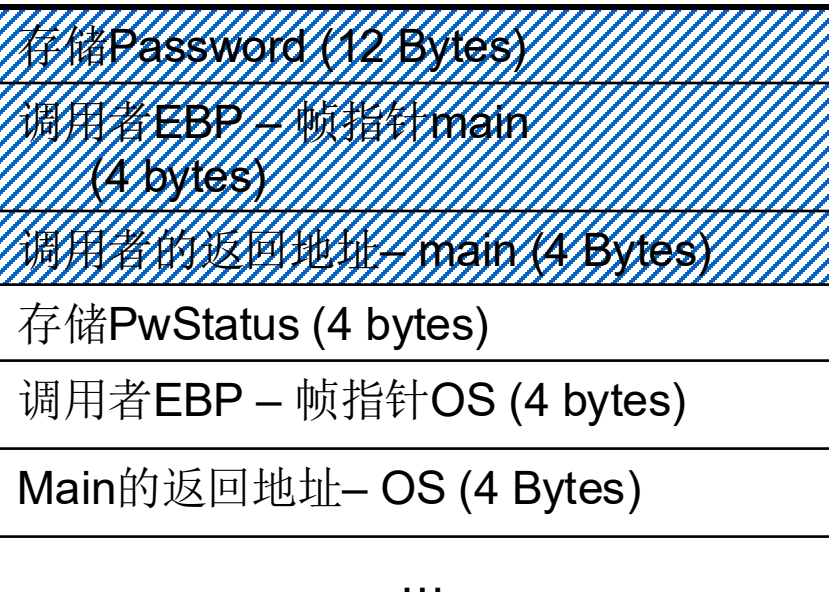
EIP



```
puts("Enter Password:");
PwStatus = IsPasswordOk();
if (PwStatus == false) {
    puts("Access denied");
    exit(-1);
}
else puts("Access granted");
```

栈

ESP



5.2 函数调用时发生了什么

我们假定有如下程序

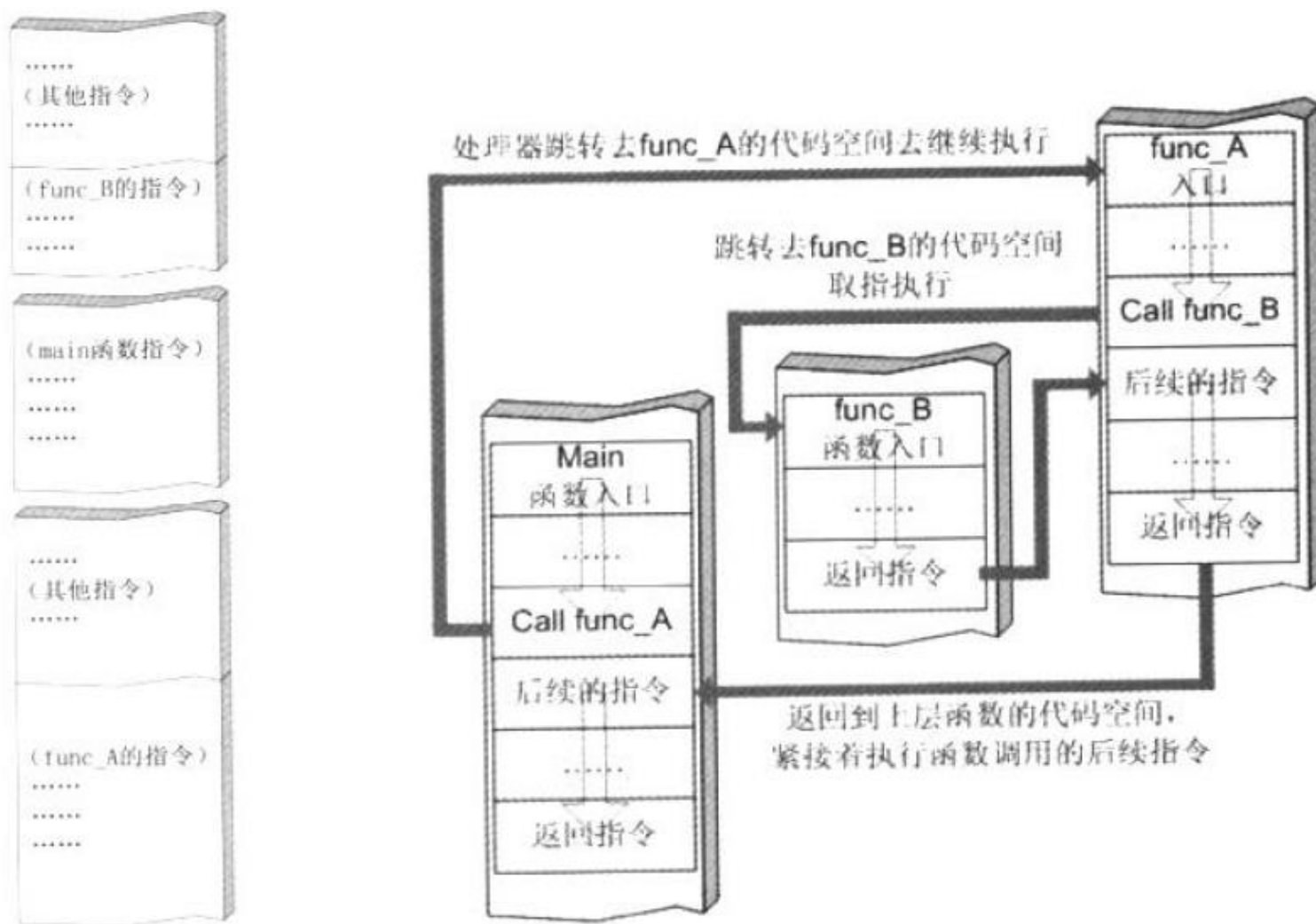
```
int funcb()  
{...  
}  
  
int funca()  
{...  
    funcb();  
...  
}
```

我们假定有如下程序

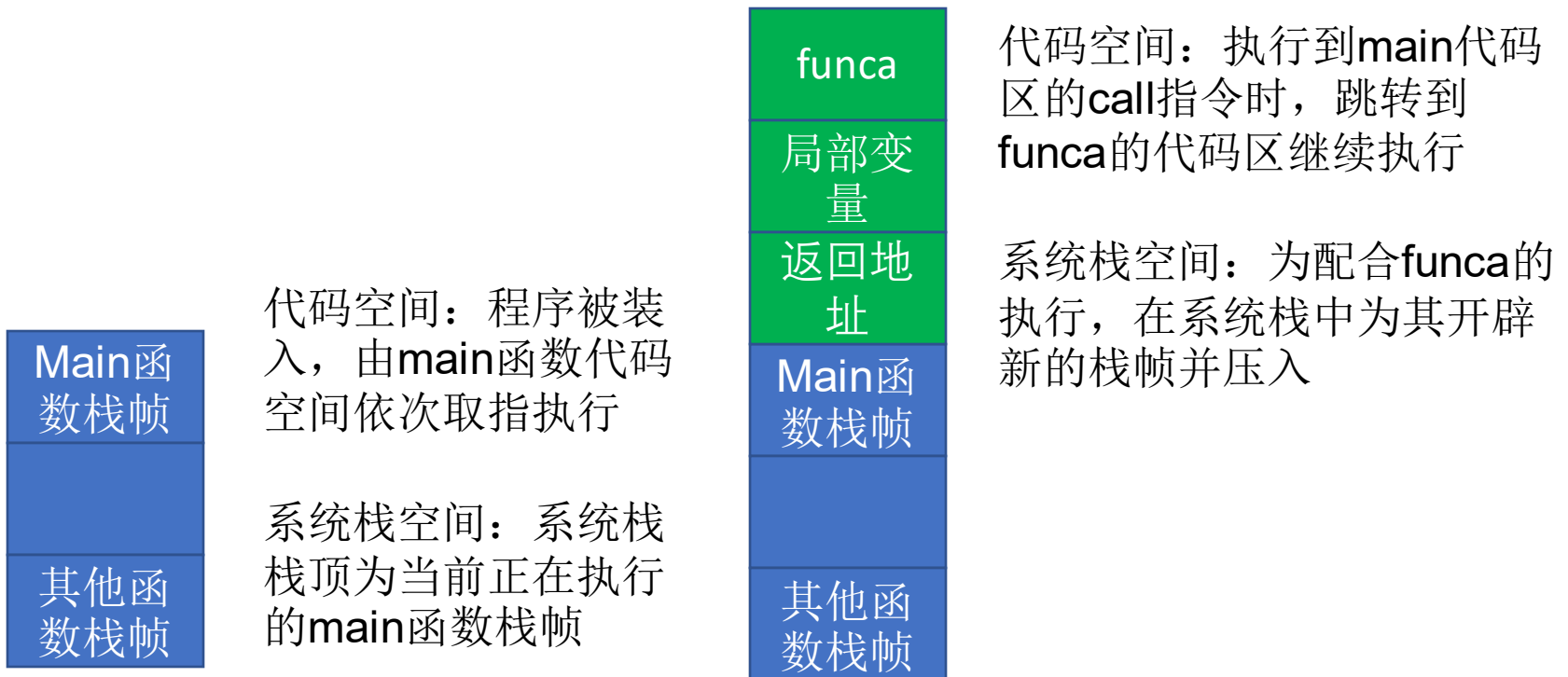
```
int main()  
{  
    funca();  
}
```

系统栈在函数调用时的变化是什么样的呢？

代码空间调用关系



系统栈在函数调用时的变化



系统栈在函数调用时的变化



代码空间：执行到 `funca` 代码区的 `call` 指令时，跳转到 `funcb` 的代码区继续执行

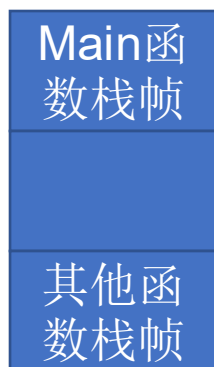
系统栈空间：为配合 `funcb` 的执行，在系统栈中为其开辟新的栈帧并压入



代码空间：`funcb` 代码执行完毕，弹出自己的栈帧并从中获得返回地址，跳回 `funca` 代码区继续执行

系统栈空间：弹出 `funcb` 的栈帧。对应于当前正在执行的函数，当前栈顶栈帧重新恢复成 `funca` 函数栈帧

系统栈在函数调用时的变化



代码空间：func_a代码执行完毕，弹出自己的栈帧并从中获得返回地址，跳回main代码区继续执行

系统栈空间：弹出func_a的栈帧。对应于当前正在执行的函数，当前栈顶栈帧重新恢复成main函数栈帧

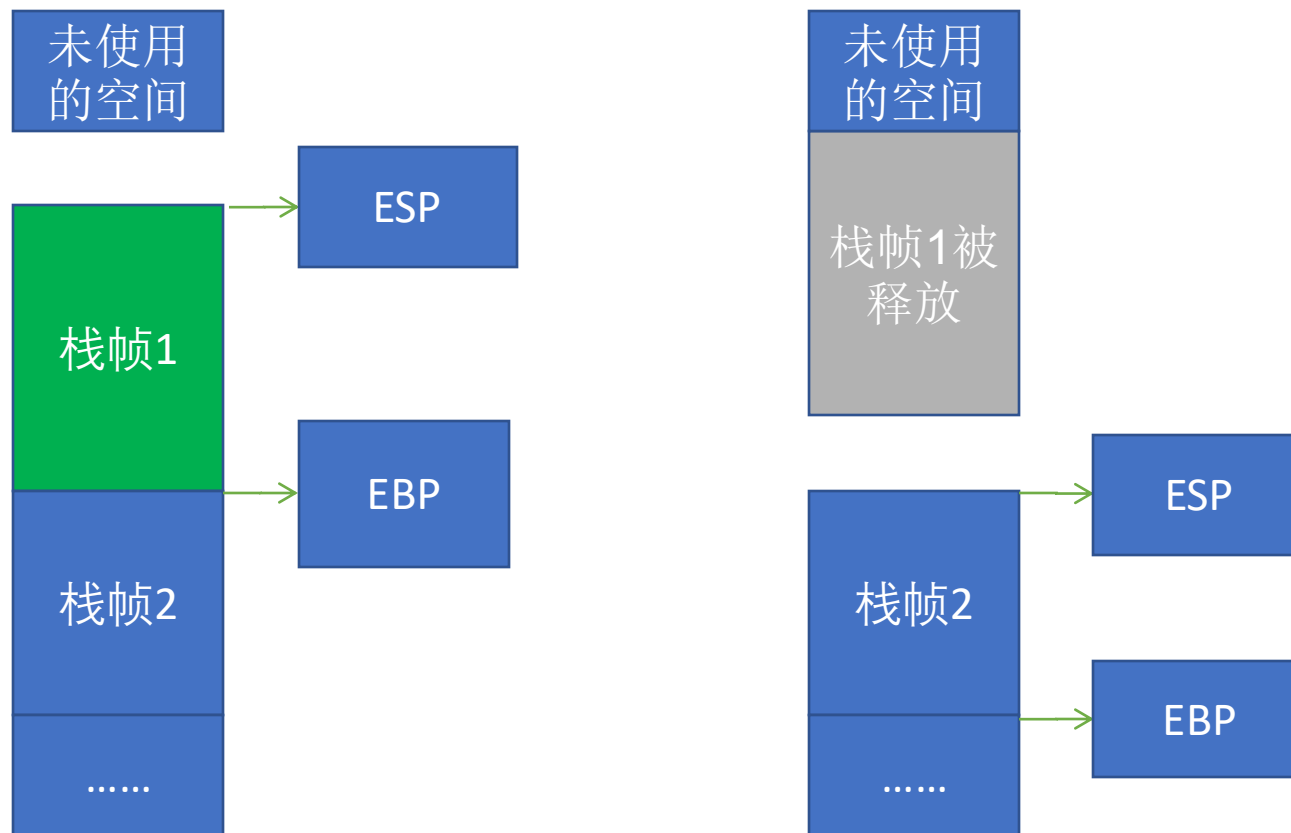
5.3 寄存器与函数栈帧

每一个函数独占自己的栈帧空间。当前正在运行的函数的栈帧总是在栈顶。Win32系统提供两个特殊的寄存器用于标识位于系统栈顶端的栈帧。

(1) ESP: 栈指针寄存器(extended stack pointer),其内存放着一个指针, 该指针永远指向系统栈最上面的一个栈帧的栈顶。

(2) EBP: 基址指针寄存器(extended base pointer),其内存放着一个指针, 该指针永远指向系统栈最上面的一个栈帧的底部。

栈帧寄存器ESP与EBP的作用



在函数栈帧中，一般包含以下几类重要信息

- (1) **局部变量**：为函数局部变量开辟的内存空间。
- (2) **栈帧状态值**：保存前栈帧的顶部和底部（实际上只保存前栈帧的底部，前栈帧的顶部可以通过堆栈平衡计算得到），用于在本帧被弹出后恢复出上一个栈帧。
- (3) **函数返回地址**：保存当前函数调用前的“断点”信息，也就是函数调用前的指令位置，以便在函数返回时能够恢复到函数被调用前的代码区中继续执行指令。

除了与栈相关的寄存器外，我们还需要记住另一个至关重要的寄存器。

EIP：指令寄存器(Extended Instruction Pointer),其内存放着一个指针，该指针永远指向一条等待执行的指令地址。

可以说如果控制了EIP寄存器的内容，就控制了进程---我们让EIP指向哪里，CPU就会去执行哪里指令。

5.4 函数调用约定与相关指令

函数调用大致包括以下几个步骤。

(1) 参数入栈：将参数从右向左一次压入系统栈中。



(2) 返回地址入栈：将当前代码区调用指令的下一跳指令地址压入栈中，供函数返回时继续执行。



(3) 代码区跳转：处理器从当前代码区跳转到被调用函数的入口处。



(4) 栈帧调整

第四步栈帧调整具体包括如下几个步骤

保存当前栈帧的状态值，以备后面恢复本栈帧时使用（EBP入栈）



将当前栈帧切换到新栈帧（将ESP值装入EBP，更新栈帧底部）



给新栈帧分配空间（把ESP减去所需空间的大小，抬高栈帧）

对于__stdcall调用约定，函数调用时用到的指令序列大致如下

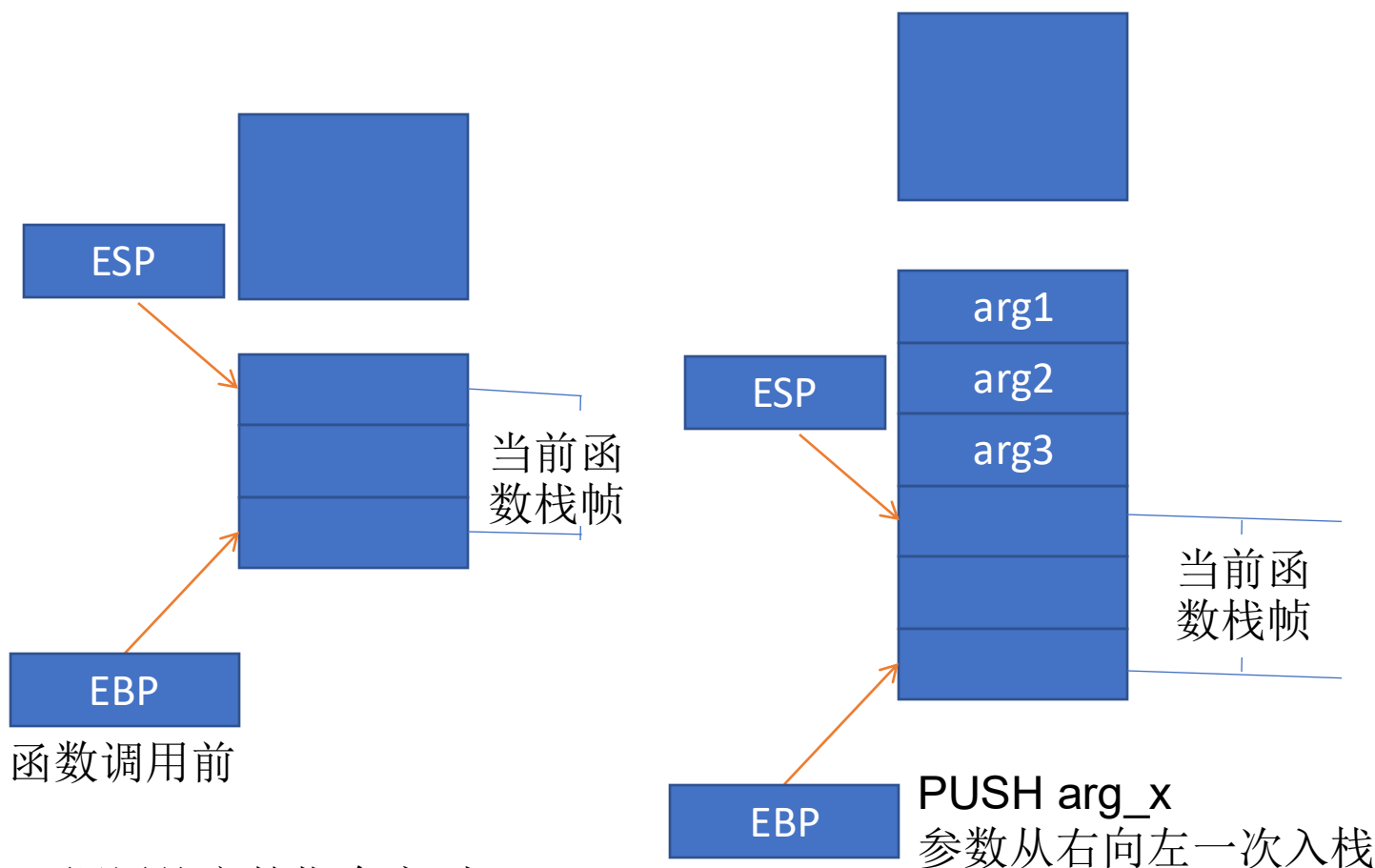
序列	备注
push 参数 3	假设该函数有3个参数，将从右向左依次入栈
push 参数 2	
push 参数 1	
call 函数地址	call指令将同时完成两项工作:a)向栈中压入当前指令在内存中的位置，即保存返回地址。 b)跳转到所调用函数的入口地址函数入口处：修改EIP
__stdcall调用约定的指令序列	

对于__stdcall调用约定，函数调用时用到的指令序列大致如下

序列	备注
push ebp	保存旧栈帧的底部
mov ebp,esp	设置新栈帧的底部(栈帧切换)
sub esp,xxx	设置新栈帧的顶部(抬高栈顶，为新栈帧开辟空间)

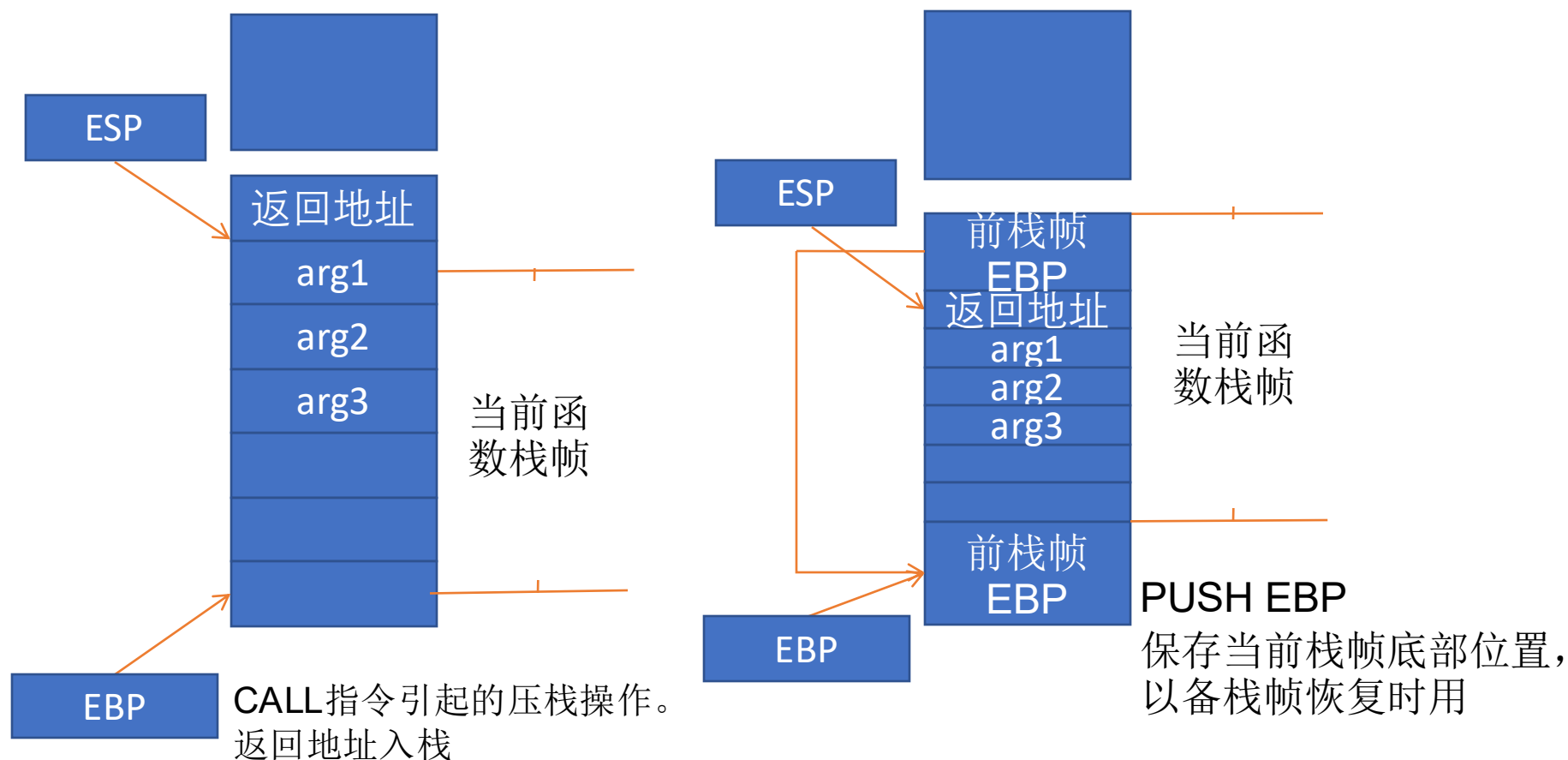
__stdcall调用约定的指令序列

函数调用时系统栈的变化过程



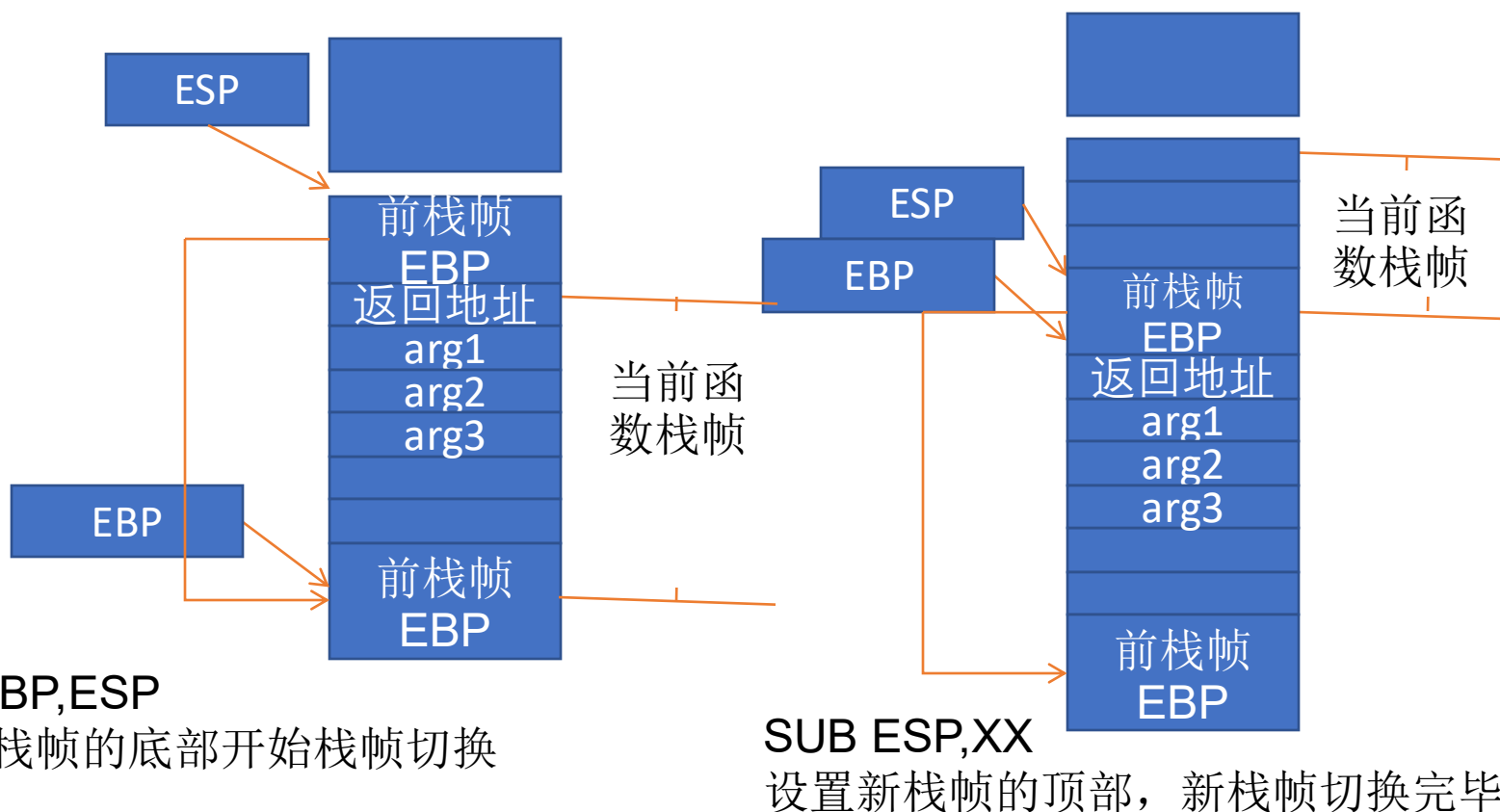
__stdcall调用约定的指令序列

函数调用时系统栈的变化过程



__stdcall调用约定的指令序列

函数调用时系统栈的变化过程



__stdcall调用约定的指令序列

类似的，函数返回的步骤如下

- (1) 保存返回值：通常将函数的返回值保存在寄存器EAX中。
- (2) 弹出当前栈帧，恢复上一个栈帧，具体包括：
 - 在堆栈平衡的基础上，给ESP加上栈帧的大小，降低栈顶，回收当前栈帧的空间。
 - 将当前栈帧底部保存的前栈帧EBP值弹入EBP寄存器，恢复上一个栈帧。
 - 将函数返回地址弹给EIP寄存器。
- (3) 跳转：按照函数返回地址跳回母函数中继续执行。

以C语言和Win32平台为例，函数返回时的相关的指令序列

指令	备注
add esp,xxx	降低栈顶，回收当前的栈帧
pop ebp	将上一个栈帧底部位置恢复到ebp
retn	这条指令有两个功能： a)弹出当前栈顶元素，即弹出栈帧中的返回地址。至此，栈帧恢复工作完成。 b)让处理器跳转到弹出的返回地址，恢复调用前的代码区

内容安排

- 软件安全基础
 - 1 计算机引导
 - 2 Win32内存体系
 - 3 PE文件结构
 - 4 进程空间分区
 - 5 系统栈工作
 - 6 缓冲区溢出

6 缓冲区溢出

- 缓冲区溢出原理简介
- 缓冲区溢出问题历史
- 缓冲区溢出的影响
- 缓冲区溢出的预防

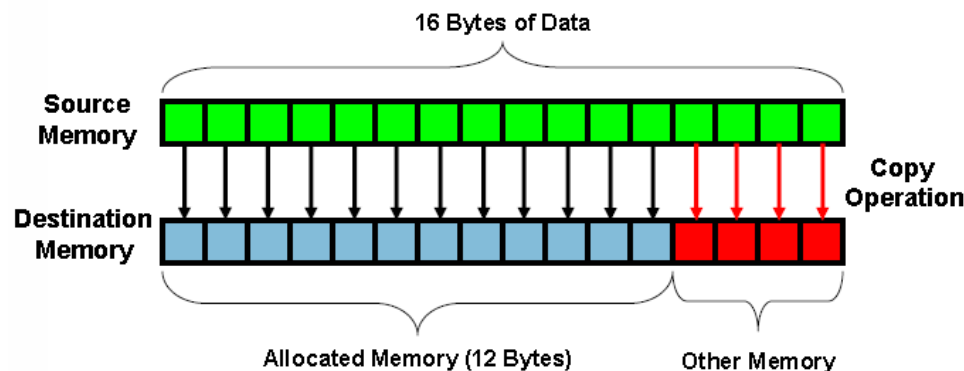
6.1缓冲区溢出原理简介

缓冲区溢出是指当计算机向缓冲区内填充数据位数时超过了缓冲区本身的容量溢出的数据覆盖在合法数据上。

这种错误的状态发生在写入内存的数据超过了分配给缓冲区的大小的时候,就像一个杯子只能盛一定量的水,如果放到杯子中的水太多,多余的水就会一出到别的地方。由于缓冲区溢出,相邻的内存地址空间被覆盖,造成软件出错或崩溃。如果没有采取限制措施,可以使用精心设计的输入数据使缓冲区溢出,从而导致安全问题。

6.1 缓冲区溢出原理简介

缓冲区溢出是最常见的内存错误之一，也是攻击者入侵系统时所用到的最强大、最经典的一类漏洞利用的方式



缓冲区溢出图示

6.2缓冲区溢出相关历史

- 很长一段时间以来,缓冲区溢出都是一个众所周知的安全问题, C程序的缓冲区溢出问题早在70年代初就被认为是C语言数据完整性模型的一个可能的后果。这是因为在初始化、拷贝或移动数据时, **C语言并不自动地支持内在的数组边界检查**。虽然这提高了语言的执行效率, 但其带来的影响及后果却是**深远和严重的**。
- 1988年 Robert T. Morris的finger蠕虫程序, 这种缓冲区溢出的问题使得Internet几乎限于停滞, 许多系统管理员都将他们的网络断开, 来处理所遇到的问题。
- 1989年Spafford提交了一份关于运行在VAX机上的BSD版UNIX的fingerd的缓冲区溢出程序的技术细节的分析报告, 引起了部分安全人士对这个研究领域的重视

6.2缓冲区溢出相关历史

- 1996年 出现了真正有教育意义的第一篇文章, Aleph One在Underground发表的论文详细描述了Linux系统中栈的结构和如何利用基于栈的缓冲区溢出。
- Aleph One的贡献还在于给出了如何写开一个shell的Exploit的方法, 并给这段代码赋予shellcode的名称, 而这个称呼沿用至今, 我们现在对这样的方法耳熟能详--编译一段使用系统调用的简单的C程序, 通过调试器抽取汇编代码, 并根据需要修改这段汇编代码。
- 1997年 Smith综合以前的文章, 提供了如何在各种Unix变种中写缓冲区溢出Exploit更详细的指导原则。

6.2缓冲区溢出相关历史

- 1998年 来自 “Cult of the Dead Cow”的Dildog在 Bugtrq邮件列表中以Microsoft Netmeeting为例子详细介绍了如何利用Windows的溢出，这篇文章最大的贡献在于提出了利用栈指针的方法来完成跳转，返回地址固定地指向地址，将Windows下的溢出 Exploit推进了实质性的一步。

6.3缓冲区溢出的影响

- 不要小看缓冲区溢出问题,它可能会产生很恶劣的影响:
 - 发布安全报告以及相关补丁的时间和费用开销
 - 数以千计的系统管理员安装补丁的时间开销
 - 系统被攻击者破坏所造成的损失
 - 重要资料被盗取造成的损失
 - 开发者的信誉....
- 粗略的算下来,一个马虎的错误就可能需要花费**几百万美元**的代价才能弥补

6.4缓冲区溢出的预防

- 面临越来越多的来自缓冲区溢出攻击的威胁,例如Immunix根据自动侦测和预防技术开发的StackGuard系统之类的侦测和防止缓冲区溢出发生的自适应技术被研发出来.防范溢出问题正受到越来越多的关注.
- 目前对于缓冲区溢出,主要分为静态保护和动态保护:
- **静态保护:**不执行代码,通过静态分析来发现代码中可能存在的漏洞.静态的保护技术包括编译时加入限制条件,返回地址保护,二进制改写技术,基于源码的代码审计等.
- **动态保护:**通过执行代码分析程序的特性,测试是否存在漏洞,或者是保护主机上运行的程序来防止来自外部的缓冲区溢出攻击.

本讲小结

- 课程介绍
- 基本概念
 - 1 软件与软件安全
 - 2 软件安全
 - 3 漏洞
 - 4 软件风险
- 软件安全基础
 - 计算机引导
 - Win32内存体系
 - PE文件结构
 - 进程空间分区
 - 系统栈工作
 - 缓冲区溢出

谢谢大家

